

**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING**

Khwopa College Of Engineering
Libali, Bhaktapur
Department of Computer Engineering



**A REPORT ON
COMPANION CARE: AN INTENTION BASED
ASSISTANCE SYSTEM USING TRANSFORMER**

Submitted in partial fulfillment of the requirements for the degree

BACHELOR OF COMPUTER ENGINEERING

Submitted by

Aayush Dahal
Jayan Ghimire
Samik Bhattarai
Sankalpa Kandel

KCE076BCT002
KCE076BCT018
KCE076BCT036
KCE076BCT040

Under the Supervision of

Er. Subhadra Joshi
Department of Computer Engineering

Khwopa College Of Engineering

Libali, Bhaktapur
2023-24

CERTIFICATE OF APPROVAL

This is to certify that this major project work entitled **COMPANION CARE: AN INTENTION BASED ASSISTANCE SYSTEM USING TRANSFORMER** submitted by Aayush Dahal (KCE076BCT002), Jayan Ghimire (KCE076BCT018), Samik Bhattarai (KCE076BCT036) and Sankalpa Kandel (KCE076BCT040) to the Department of Computer Engineering in partial fulfillment of requirement for the degree of Bachelor of Engineering in Computer Engineering. The project was carried out under special supervision and within the time frame prescribed by the syllabus.

.....
Er. Bibha Sthapit
External Examiner
Assistant Professor
Department of Electronics and
Computer Engineering
IOE, Pulchowk

.....
Er. Subhadra Joshi
Project Supervisor
Lecturer
Department of Computer Engineering
Khwopa College of Engineering

.....
Er. Dinesh Gothe
Head of Department,
Department of Computer Engineering
Khwopa College of Engineering

Acknowledgement

This project is prepared in partial fulfilment of the requirement for the bachelor's degree in Computer Engineering. We owe our deepest gratitude to the Department of Computer Engineering, Khwopa College of Engineering for providing us with an opportunity to work on a major project as a part of our syllabus. Our sincere thanks go to our supervisor, Er. Subhadra Joshi, for her invaluable guidance, encouragement, and unwavering support throughout the course of this project. We would also like to owe our gratitude to our department head Er. Dinesh Gothe for his guidance. The experience of working on this project will surely enrich our technical knowledge and also gives experience of working on a project and also develop our team works skills to a great extent.

Thank you.

Aayush Dahal	KCE076BCT002
Jayan Ghimire	KCE076BCT018
Samik Bhattarai	KCE076BCT036
Sankalpa Kandel	KCE076BCT040

Abstract

The rising prevalence of mental health issues has highlighted the urgent need for accessible and scalable solutions to support individuals in maintaining their mental well-being. In this project, we developed an intention based assistance system that harnesses the power of a transformer-based model. By leveraging advancements in natural language processing and machine learning, our chatbot employs a textual and audio based conversational interface to foster empathetic and non-judgmental interactions with users. This project deals with generative model which is trained on a large dataset of conversational data and uses attention mechanisms to learn the relationships between words and their meaning. Our chatbot integrates Voice and Text Emotion Recognition, Intent Recognition, and a Language Model for user responses. This combination enables the chatbot to understand emotions in spoken and written language, accurately identify user intents, and generate contextually relevant replies. In voice emotion recognition, we leverage the efficiency of a 1D-CNN algorithm. For text emotion recognition, the robust BERT model is utilized. Intent recognition is achieved through the application of the SVM algorithm, ensuring accurate identification of user intentions. The generative model, responsible for dynamic responses, is powered by the Transformer algorithm. The effectiveness of the various models used are evaluated through various metrics. Overall, this project demonstrates the potential of using transformer models for building chatbots that can understand human language and provide intelligent responses.

Keywords: *Machine learning, Transformer, Embedding, Tokens, Chatbot, Generative Model, Deep Learning, Convolution Neural Network*

Contents

Certificate of Approval	i
Acknowledgement	ii
Abstract	ii
List of Tables	vi
List of Figures	viii
List of Symbols and Abbreviation	x
1 Introduction	1
1.1 Background Introduction	1
1.2 Problem Statement	2
1.3 Obejective	2
2 Literature Review	3
3 Requirement Analysis	9
3.1 Functional Requirements	9
3.2 Non Functional Requirements	9
3.2.1 Performance	9
3.2.2 Usability	9
3.2.3 Maintainability	9
3.3 Tools Used	10
3.3.1 Hardware Requirements	10
3.3.2 Software Requirements	10
3.4 Feasibility Study	10
3.4.1 Economic Feasibility	10
3.4.2 Technical Feasibility	10
3.4.3 Operational Feasibility	11
3.4.4 Schedule Feasibility	11
4 System Design and Architecture	12
4.1 Use Case Diagram	12
4.2 System Architecture	13
5 Methodology	14
5.1 Software Development Model	14
5.1.1 Iterative Model	14
5.2 Models	15
5.2.1 Voice Emotion Recognition	16

5.2.1.1	Feature Extraction	16
5.2.1.2	Convolution Neural Network	16
5.2.2	Text Emotion Recognition	17
5.2.2.1	BERT	17
5.2.2.2	Adam Optimizer	18
5.2.2.3	Categorical Cross Entropy	20
5.2.3	Intents Recognition	20
5.2.3.1	TF-IDF	20
5.2.3.2	Support Vector Machines	21
5.2.3.3	AdaBoost	22
5.2.4	Gradient Boosting	23
5.2.5	Language Model	24
5.2.5.1	Transformer	24
5.2.5.2	Sequence to Sequence Model	27
5.3	Evaluation Metrics	29
5.3.1	Accuracy	29
5.3.2	Precision	29
5.3.3	Recall	30
5.3.4	F1-Score	30
5.3.5	Learning Curve	30
5.3.6	Perplexity	30
5.4	Model Training	31
5.4.1	Voice Emotion Recognition Model	31
5.4.1.1	Data Collection	31
5.4.1.2	Visualization of Audio files	31
5.4.1.3	1d-CNN	34
5.4.2	Text Emotion Recognition Model	36
5.4.2.1	Data Collection	36
5.4.2.2	Text Preprocessing	36
5.4.2.3	Model Loading	37
5.4.3	Intent Recognition Model	38
5.4.3.1	Data Collection	38
5.4.3.2	Data Visualization	39
5.4.3.3	Text Preprocessing	39
5.4.3.4	Feature Extraction	39
5.4.3.5	Model Building	40
5.4.4	Language Model	41
5.4.4.1	Data Collection	41
5.4.4.2	Seq2Seq	42
5.4.4.3	Transformer Architecture	43
5.5	UI Development	46
6	Result and Discussion	48
6.1	Voice Emotion Detection Model	48
6.2	Text Emotion Detection Model	49
6.3	Intention Detection Model	50

6.4	Language Model	53
7	Conclusion	55
8	Limitations and Future Enhancements	56
8.1	Limitations	56
8.2	Future Enhancements	56
	Bibliography	60
	Appendix	61
C	User Interface	61
C.1	Home Page	61
C.2	Register Page	63
C.3	Chat Demo	65

List of Tables

2.1	Literature Review Matrix	5
5.1	Sample DataSet	38
5.2	Category Counts	38
5.3	Sample Data with Cleaned Messages and Topic Encoding	39
6.1	Classification Report for Voice Emotion Recognition Model	49
6.2	Classification Report for Text Emotion Recognition Model	49
6.3	Classification Report for a Adaboost Classifier	51
6.4	Classification Report for a Gradient Boosting	51
6.5	Classification Report for SVM	52

List of Figures

4.1	System Use Case Diagram	12
4.2	System Architecture	13
5.1	Iterative Model for Software Development	14
5.2	Simple 1D-CNN	17
5.3	BERT for emotion classification	18
5.4	Support Vector Machine	22
5.5	AdaBoost Classifier	23
5.6	Gradient Boosting	24
5.7	Transformer	25
5.8	Scaled dot product attention	26
5.9	Feed-Forward Network	27
5.10	Illustration of a Seq2Seq Model with Attention	28
5.11	Emotion Counts	31
5.12	Spectrogram for Audio with Fear Emotion	31
5.13	Waveplot for Audio with Fear Emotion	32
5.14	Spectrogram for Audio with Angry Emotion	32
5.15	Waveplot for Audio with Angry Emotion	32
5.16	Spectrogram for Audio with Disgust Emotion	32
5.17	Waveplot for Audio with Disgust Emotion	33
5.18	Spectrogram for Audio with Neutral Emotion	33
5.19	Waveplot for Audio with Neutral Emotion	33
5.20	Spectrogram for Audio with Sad Emotion	33
5.21	Waveplot for Audio with Sad Emotion	34
5.22	Spectrogram for Audio with Happy Emotion	34
5.23	Waveplot for Audio with Happy Emotion	34
5.24	Model Summary for Voice Emotion Recognition	35
5.25	Text Emotion Model Data-Set Sample	36
5.26	Model Summary for Text Emotion Recognition Model	37
5.27	Data Visualization	39
5.28	Block Diagram of seq2seq Architecture	42
5.29	Model Summary for Seq2Seq Architecture	43
5.30	Seq2Seq Model Description	43
5.31	Model Summary for Custom Transformer Architecture	45
5.32	Custom Transformer Architecture	46
6.1	Accuracy and Loss Curve for Voice Emotion Recognition Model	48

6.2	Confusion matrix for Voice Emotion Recognition Model	48
6.3	Confusion Matrix for Text Emotion Recognition Model	50
6.4	Confusion matrix for SVM	52
6.5	Accuracy Curve for Seq2Seq Model	53
6.6	Accuracy Curve for Transformer Model	54

List of Symbols and Abbreviation

API	Application Protocol Interface
BERT	Bidirectional Encoder Representations from Transformers
BRNN	Bidirection Recurrent Neural Network
CNN	Convolution Neural Network
JS	JavaScript
LLM	Large Language Model
LSTM	Long Short-Term Memory
NLP	Natural Language Processing
RMSP	Root Mean Square Propagation
RNN	Recurrent Neural Network
SVM	Support Vector Machine
UML	Unified Modeling Language
WAA	Weighted Average Accuracy

Chapter 1

Introduction

1.1 Background Introduction

"Our attitude towards life determines life's attitude towards us." [1] this motivational quote tells us to live life with a positive intent. It elaborates the idea that if we live life with positive intention every thing around us will be positive. In today's modern time despite having many online friends we barely have some one to talk to leading to loneliness. Having someone to talk share their feeling is something we lack in modern days.

Loneliness, lack of proper social communication is something that leads to weak mental health. Having a healthy physical body is very important but to support that a sound mind is equally important. Having someone to talk to prevents depression, anxiety and feeling of loneliness. In recent years, there has been increasing acknowledgement that mental health plays important role in achieving global development goals. Depression is one of the leading causes of disability. Suicide is the fourth leading cause of death among 15-29 year-old's. [2] People with severe mental health conditions die prematurely – as much as two decades early – due to preventable physical conditions.

In Nepal, according to a National Mental Health Survey in 2020, nearly 80 percent of people who need mental health care do not get it. According to data obtained from the Nepal police, the average annual increase in suicides over the past five years in Nepal is 7.2% which increased to 14% in 2021. [3] This has resulted in Nepal government coming up with its own suicide hot line to prevent suicide related deaths from increasing.

Our intention meaning our thoughts and behaviour determines our mood happy, angry, fear, confuse. It affects our action accordingly how we handle different situation. How we handle stress how to decide to act wisely in difficult situation or simply to differentiate between whats wrong or whats right. The tone of speech plays a crucial role in conveying the emotion of a person. It encompasses the pitch, volume, pace, and intonation with which words are spoken. A warm and gentle tone typically signifies friendliness or happiness, while a sharp or loud tone may

suggest anger or frustration. The modulation and rhythm of speech can convey enthusiasm, sadness, excitement, or boredom. Additionally, the choice of words, emphasis on certain phrases, and overall contribute to the emotional undertones. The tone provides listeners with valuable cues to interpret the speaker's feelings, allowing for a more nuanced understanding of the intended message beyond the literal words spoken.

Having someone to talk to is vital not only for the younger generation but also for the elderly. In the context of the increasing trend of foreign employment, older individuals may find themselves with even fewer people to engage in meaningful conversations. Post-retirement, when the regular social interactions associated with the workplace diminish, there is a heightened risk of loneliness and isolation among the elderly. Social connections are essential for mental and emotional well-being, and without regular communication, older individuals may face challenges in maintaining a sense of purpose and belonging. Loneliness in the elderly has been linked to various health issues, including depression and cognitive decline. Therefore, fostering opportunities for social interaction and providing support networks for the elderly is crucial in addressing the emotional well-being of this demographic, ensuring they lead fulfilling and connected lives even in their later years.

1.2 Problem Statement

In today's digitally connected world, individuals seek personalized support and companionship that aligns with their preferences and lifestyles. However, traditional methods of accessing such support, whether for entertainment or guidance, can be limited by factors like cost, availability, and stigma. As a result, there is a growing need for innovative solutions that cater to diverse needs in a convenient and accessible manner.

Addressing this need involves developing an intention-based chatbot system capable of engaging users in empathetic conversations, offering personalized recommendations, and providing entertainment such as jokes and music suggestions. By leveraging advanced technologies, the aim is to create a user-centric platform that not only fosters meaningful interactions but also promotes overall well-being by catering to individual preferences and needs. A chatbot can bridge the gap between traditional support systems and modern digital solutions, empowering individuals to navigate life's challenges with confidence and ease.

1.3 Objective

The main aim of this project is:

- To create a voice based personalized chatbot system offering support, companionship and entertainment to users, enhancing overall well-being.

Chapter 2

Literature Review

In this paper [4], SERMO engages users on a daily basis, inquiring about events and emotions they have experienced. Through natural language processing and a lexicon-based approach, the chatbot automatically identifies the user’s basic emotion from their input. Based on the identified emotion, SERMO suggests appropriate measures such as activities or mindfulness exercises to help users regulate their emotions and cope with thoughts and feelings.

In this paper [5], the authors introduce Serena, a deep learning (DL) dialogue system specifically developed for mental health counseling. Serena consists of a core generative model, a Seq2Seq Transformer with 2.7 billion parameters, which has been fine-tuned using transcripts of person-centered therapy (PCT) sessions. The system also incorporates post-processing algorithms to detect contradictions, enhance coherency, and eliminate repetitive answers.

This paper [6] aims to develop a mental health chatbot that offers emotional support and therapy to users. The chatbot utilizes a logistic regression model for emotion and sentiment analysis to understand the user’s emotions and provide relevant suggestions. It also incorporates the Kessler’s Psychological Distress Scale to assess the user’s mental state and deliver appropriate responses. By combining these features, the chatbot offers personalized and targeted support, enhancing its ability to assist users in managing their mental health.

This paper [7] discusses the limitations of early studies on using chatbots for psychiatric counseling and proposes an improved approach. The authors emphasize the importance of considering the user’s psychiatric status, continuous monitoring, accurate emotion recognition, and ethical judgment in interventions. To address these aspects, they suggest a conversational service for psychiatric counseling that incorporates high-level natural language understanding and a multi-modal approach to emotion recognition. By combining these methodologies, the proposed approach aims to enable sensitive and continuous observation of emotional changes in users, while providing appropriate and ethical responses in clinical psychiatric counseling.

This paper [8] investigates user engagement and interaction patterns with a chatbot called Tess, designed to address depression. The aim is to gain insights into how users utilize the chatbot and provide design recommendations. The analysis involved 354 users, examining their interactions and usage across different depression modules. The findings reveal that users sent a significant number of messages and engaged with the chatbot for an average of 46 days. However, there was considerable heterogeneity in user engagement, influenced by factors such as module length, complexity, content, question style, and routing between modules. The study emphasizes the importance of module design and offers implications for future chatbot development and evaluation.

This paper [9] uses a 1D Convolutional Neural Network (1D CNN) architecture with dilated convolutions, along with Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) layers for the SER task. The proposed models are trained on five different datasets which utilizes different spectral features: MFCC, LMS, ZCR, Chromagram, and RMS values of the speech audio files as the input. Weighted average accuracy of 99.46 with augmented data is obtained in TESS dataset and WAA of 90.47 is obtained with augmented data in CREMA-D datasets.

The model used in this paper [10] is a combination of CNNs, attention mechanisms, and fully connected layers, where CNNs are used to extract features from the input data, the attention mechanisms are used to focus on the most important features, and the fully connected layers are used to classify the data from IEMOCAP and EMO-DB datasets. This model is designed to address the challenges of speech emotion recognition, such as the loss of important information due to segmentation and the need for generalization across different lengths of speech spectrograms. The model gave the average accuracy of 89.76 in EMO-DB datasets and 91.18 in IEMOCAP datasets.

This paper [11] uses CNN model for SER which consists of three 1D convolutional layers (CLs), each followed by a dropout, activation, and max-pool layer, where first CL receives an array of features with a 1-pixel stride, 64 filters, and a 5-size kernel and output of every neuron is activated using ReLU activation function, followed by a dropout rate of 0.2. The proposed technique achieved an accuracy of 96.6 for the EMO-DB datasets, 92.6 for the SAVEE dataset, and 91.4 for the RAVDESS dataset. The model is a novel fusion of MFCCs and t-domain features coupled with a 1D CNN for emotion classification which aims to enhance the accuracy and robustness of speech emotion recognition (SER) systems.

This paper [12] proposes a deep learning-assisted semantic text analysis (DLSTA) model for human emotion detection using big data and the proposed approach expressly achieves the highest detection rate of 97.22 and 98.02 of classification accuracy with various emotional term embedding methods. DLSTA demonstrates superior performance in terms of prediction accuracy, classification accuracy, detection rate, precision, performance, and recall ratio compared to existing methods.

Table 2.1: Literature Review Matrix

S.N	Title	Authors	Features/Limitations
1	A Mental Health Chatbot for Regulating Emotions (SERMO) - Concept and Usability Test [4]	Kerstin De-necke,Sayan Vaaheesan and Aaganya Arul-nathan	• The chatbot automatically identifies the user's basic emotion from their input • A usability test is conducted to study the user experience and quality of the app and to determine areas of improvement
2	2023 IEEE International Conference on Big Data and Smart Computing (Big-Comp) [5]	Brocki, Lennart and Dyer, George C. and Gładka, Anna and Chung, Neo Christopher	• Serena, a deep learning (DL) dialogue system specifically developed for mental health counseling. • Model system incorporates post-processing algorithms to detect contradictions,enhance coherency,and eliminate repetitive answers.
3	Mental Health Assist and Diagnosis Conversational Interface using Logistic Regression Model for Emotion and Sentiment Analysis [6]	S Moulya and T R Pragathi	• The chatbot utilizes a logistic regression model for emotion and sentiment analysis to understand the user's emotions and provide relevant suggestions • It incorporates the Kessler's Psychological Distress Scale to assess the user's mental state and deliver appropriate responses

4	A Chatbot for Psychiatric Counseling in Mental Healthcare Service Based on Emotional Dialogue Analysis and Sentence Generation [7]	Oh, Kyo-Joong and Lee, Dongkun and Ko, Byungsoo and Choi, Ho-Jin	• It suggests a conversational service for psychiatric counseling that incorporates high-level natural language understanding and a multi-modal approach to emotion recognition • Proposed approach aims to enable sensitive and continuous observation of emotional changes in users
5	Artificial Intelligence Chatbot for Depression: Descriptive Study of Usage [8]	Dosovitsky, G. and Pineda, B. S. and Jacobson, N. C. and Chang, C. and Bunge, E. L.	• Engagement and interaction patterns with a chatbot called Tess, designed to address depression • Emphasizes the importance of module design and offers implications for future chatbot development and evaluation
6	An ensemble 1D-CNN-LSTM-GRU model with data augmentation for speech emotion recognition [9]	Ahmed, Md Rayhan and Islam, Salekul and Islam, AKM Muzahidul and Shatabda, Swakkhar	• Model is trained on five different datasets which utilizes different spectral features: MFCC, LMS, ZCR, Chromagram, and RMS values of the speech audio files as the input. • Weighted average accuracy of 99.46 with augmented data is obtained in TESS dataset and WAA of 90.47 is obtained with augmented data in CREMA-D datasets.

7	Modeling Speech Emotion Recognition via Attention-Oriented Parallel CNN Encoders [10]	Makhmudov, Fazliddin and Kutlimuratov, Alpamis and Akhmedov, Farkhod and Abdallah, Mohamed S and Cho, Young-Im	• Designed to address the challenges of speech emotion recognition, such as the loss of important information due to segmentation and the need for generalization across different lengths of speech spectrograms • Model gave the average accuracy of 89.76 in EMO-DB datasets and 91.18 in IEMO-CAP datasets
8	Speech Emotion Recognition through Hybrid Features and Convolutional Neural Network [11]	Alluhaidan, Ala Saleh and Saidani, Oumaima and Jahangir, Rashid and Nauman, Muhammad Asif and Neffati, Omnia Saidani	• Model is a novel fusion of MFCCs and t-domain features coupled with a 1D CNN for emotion classification which aims to enhance the accuracy and robustness of speech emotion recognition (SER) systems • Proposed technique achieved an accuracy of 96.6 for the EMO-DB datasets, 92.6 for the SAVEE dataset, and 91.4 for the RAVDESS dataset.

9	Deep learning approach to text analysis for human emotion detection from big data [12]	Guo, Jia	• This model demonstrates superior performance in terms of prediction accuracy, classification accuracy, detection rate, precision and recall ratio compared to existing methods. • Provides the highest detection rate of 97.22 and 98.02 of classification accuracy with various emotional term embedding methods.
---	--	----------	---

Our chatbot model integrates state-of-the-art techniques identified in the literature review to create an advanced emotion-based conversational agent. Drawing upon insights from various studies, our model incorporates a 1D CNN for voice emotion recognition, BERT for text emotion analysis, SVM for intention recognition, and a Transformer-based generative model for response generation. Our chatbot incorporates assessments of users' mental states and continuously adapts its responses based on observed emotional changes. Through rigorous usability testing, we ensure the quality and effectiveness of our chatbot, identifying areas of improvement and striving for a user-centric design that prioritizes empathetic and helpful interactions.

Chapter 3

Requirement Analysis

3.1 Functional Requirements

The function requirements for the system are listed below:

- The system can provide contextually appropriate response's to the user input by employing various cutting edge technologies like text emotion recognition, voice emotion recognition, intention detection and language models.
- The system provides facility for new users to sign up in the platform
- The system provides facility for already registered user to sign in and use the platform

3.2 Non Functional Requirements

3.2.1 Performance

The system provides contextually appropriate response's to the user input with minimum delay

3.2.2 Usability

The system interface is self-explanatory and easy to use and navigate for the end user

3.2.3 Maintainability

The system is designed using the principle of modular design making it easy to maintain and also to add new features in future

3.3 Tools Used

3.3.1 Hardware Requirements

Given the fact that this is a software based project any external hardware was not employed. The models were trained in PC with GPU computing power. The designing of interface was carried out in PC with simple CPU computing power.

3.3.2 Software Requirements

The various software employed during the development of project are listed below:

- HTML
- CSS
- Javascript
- ExpressJS
- NodeJS
- Github
- StarUML
- Jupyter Notebook
- Visual Studio Code
- Google Colaboratory
- Latex

3.4 Feasibility Study

3.4.1 Economic Feasibility

Economically the cost for building the system not very significant. With no hardware requirements for this project we were able to complete the project without any overwhelming financial overhead. All the datasets we used were light weight which allowed us the flexibility of using our own PC's for training the models using the dataset which contributed to the project being economically viable. Also storing these models was not a problem because they were lightweight which also did not demand any additional hardware purchase for storing the models.

3.4.2 Technical Feasibility

Since the system implementation demanded no significant hardware or software requirements it was technically feasible to implement it

3.4.3 Operational Feasibility

In our initial objective we aimed to design and implement a robust and easy to use intent based assistance system which this system satisfies so yes the system is operationally feasible

3.4.4 Schedule Feasibility

The ready to use system was built with the preplanned schedule so the system is deemed feasible in terms of schedule

Chapter 4

System Design and Architecture

4.1 Use Case Diagram

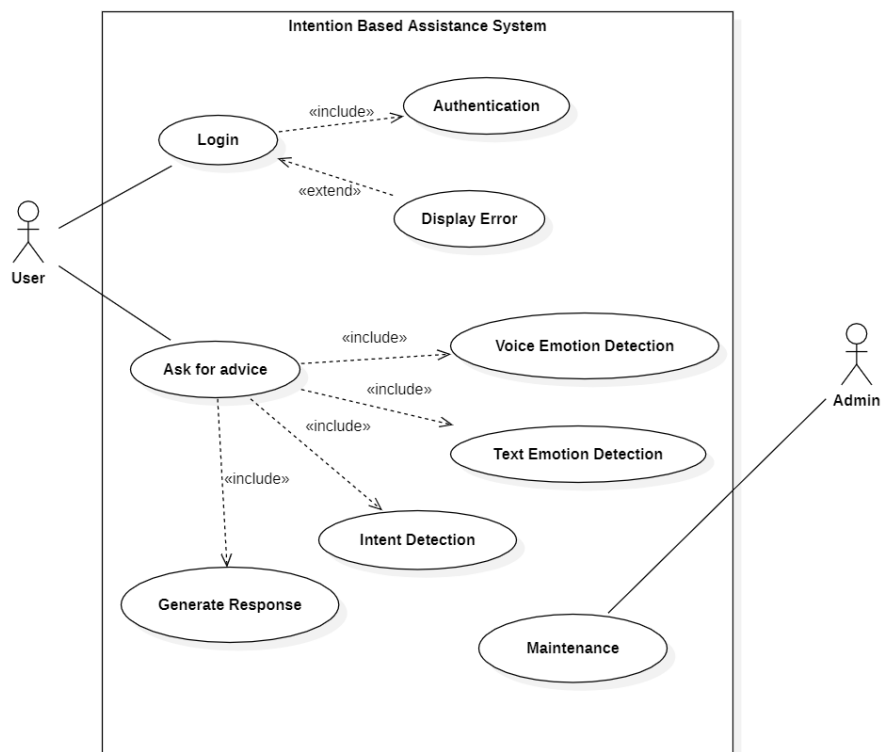


Figure 4.1: System Use Case Diagram

4.2 System Architecture

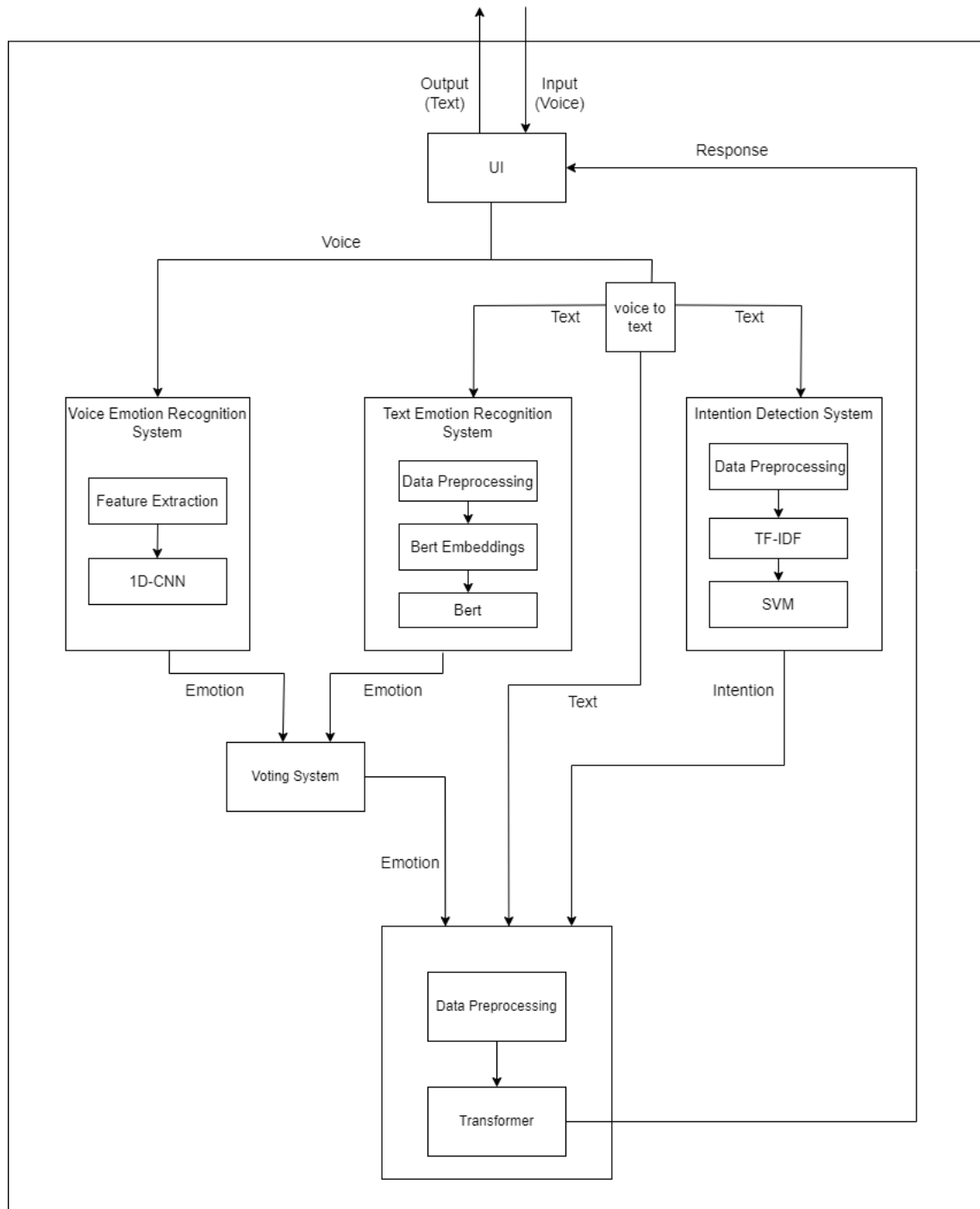


Figure 4.2: System Architecture

Chapter 5

Methodology

5.1 Software Development Model

5.1.1 Iterative Model

In adopting the iterative model for our software development, we implemented a structured approach where each iteration involved comprehensive design, implementation, testing, and review phases. Through these repetitive cycles, we sought to refine our product incrementally. The main idea was that we were able to develop the system in smaller portions through iterations. Our project was completed in a total of 3 iterations. The activities carried out during the various iterations are listed below:

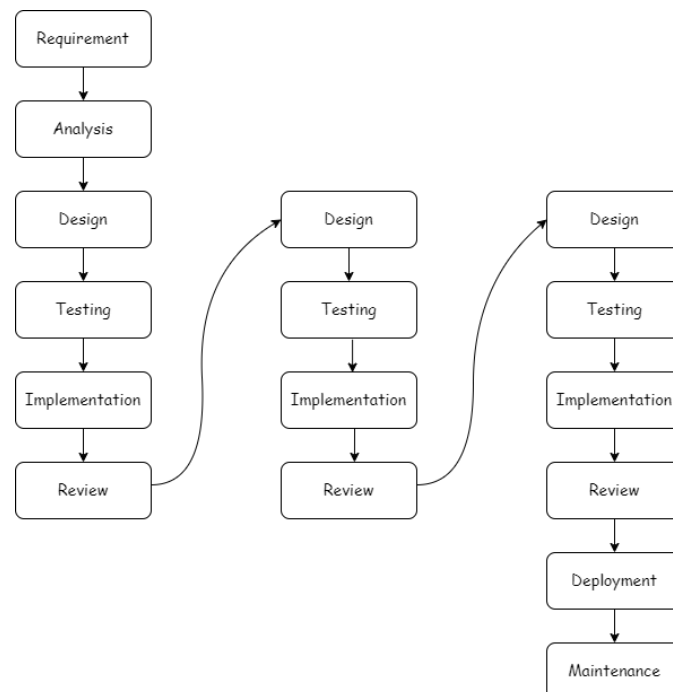


Figure 5.1: Iterative Model for Software Development

- Iteration 1

The various activities carried out during first iteration are listed below:

- Defining an high level idea about the workflow about how the system should work
- Started researching and experimenting about various Voice emotion detection models
- Started researching and experimenting with various Speech emotion detection models

- Iteration 2

The various activities carried out during the second iteration are listed below:

- After extensive research and experimentation was carried out in previous iteration regarding the text and speech emotion recognition we came up with the best performing models for them and integrated them
- Started research and experimentation with the various Intent detection and Generative models
- Started the development of user interface

- Iteration 3

The various activities carried out during the third iteration are listed below:

- After extensive research and experimentation was carried out in previous iteration regarding the Intent recognition and generative models we came up with the best performing models for them and integrated them
- All the models were integrated and the resulting system was tested
- Finally the models were integrated with the user interface developed

5.2 Models

Our chatbot utilizes a combination of advanced natural language processing (NLP) and machine learning (ML) models to accurately recognize emotions from both speech and text inputs, as well as infer user intentions. For emotion detection from speech, we employ state-of-the-art speech processing models such as convolutional neural networks (CNNs) trained on large datasets of annotated audio samples. Similarly, for text-based emotion recognition, our system leverages BERT Pre-trained model fine tuned on custom emotion dataset. Additionally, our chatbot employs intent recognition models, including deep learning-based classifiers, to understand the user's underlying purpose or goal. By combining these cutting-edge techniques, our chatbot delivers contextually appropriate responses using our custom transformer based architecture tailored to the user's emotional state and intentions.

5.2.1 Voice Emotion Recognition

5.2.1.1 Feature Extraction

We have used following method in order to extract features from the audio signals:

- Zero Crossing Rate: The rate of sign-changes of the signal during the duration of a particular frame.
- Energy: The sum of squares of the signal values, normalized by the respective frame length.
- Entropy of Energy: The entropy of sub-frames' normalized energies. It can be interpreted as a measure of abrupt changes.
- MFCCs: Mel Frequency Cepstral Coefficients form a cepstral representation where the frequency bands are not linear but distributed according to the mel-scale.

5.2.1.2 Convolution Neural Network

A one-dimensional Convolutional Neural Network (1D CNN) is a type of neural network architecture commonly used for processing and analyzing sequential data. While traditional CNNs are well-known for their effectiveness in tasks like image recognition, 1D CNNs are specifically designed to handle one-dimensional sequential data such as time series data, audio signals, textual data, and genomic sequences. The elements of 1D CNN are:

1. Input Layer: The input to a 1D CNN is typically a one-dimensional sequence of data, such as a audio features or a sequence of words in a sentence.
2. Convolutional Layers: Convolutional layers consist of filters that slide across the input data to detect patterns or features. In the context of a 1D CNN, the filters slide along the sequence in one dimension. Each filter extracts different features from the input data by performing convolution operations. The number of filters and their sizes are hyperparameters that can be adjusted based on the complexity of the problem.
3. Activation Function: After each convolution operation, an activation function like ReLU (Rectified Linear Unit) is applied to introduce non-linearity into the model, allowing it to learn complex patterns.
4. Pooling Layers: Pooling layers down-sample the features extracted by the convolutional layers, reducing the spatial dimensions of the data while retaining important information. Max pooling is commonly used in 1D CNNs, where the maximum value within each window is retained, discarding the rest.
5. Flattening Layer: Following the convolutional and pooling layers, a flattening layer reshapes the output into a one-dimensional vector, preparing it to be fed into the fully connected layers.

6. Fully Connected Layers: These layers process the flattened features to learn complex relationships and patterns in the data. The output of the last fully connected layer is often fed into a softmax layer for classification tasks or a regression layer for regression tasks.
7. Output Layer: The output layer produces the final predictions based on the task at hand, such as classification labels or numerical values.

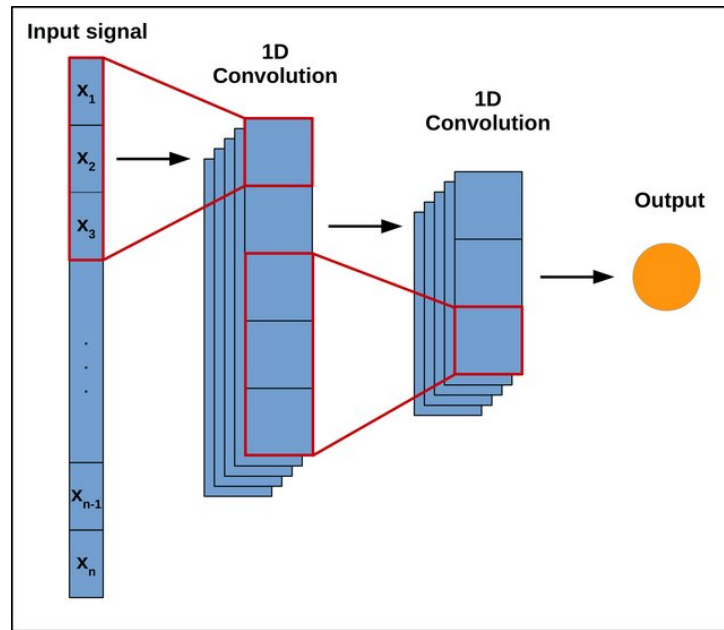


Figure 5.2: Simple 1D-CNN
Source: [Research Gate \[13\]](#)

5.2.2 Text Emotion Recognition

In the realm of Natural Language Processing (NLP), understanding and deciphering human emotions from text have emerged as pivotal challenges. Text emotion detection, a subfield of NLP, aims to discern and categorize the emotional nuances embedded within textual content. This capability finds diverse applications, ranging from sentiment analysis in customer reviews to enhancing virtual assistants' comprehension of user sentiments. One of the groundbreaking advancements in the field of NLP is the introduction of BERT, or Bidirectional Encoder Representations from Transformers.

5.2.2.1 BERT

The figure below gives a brief over-view about how we used BERT in the context of text emotional Recognition:

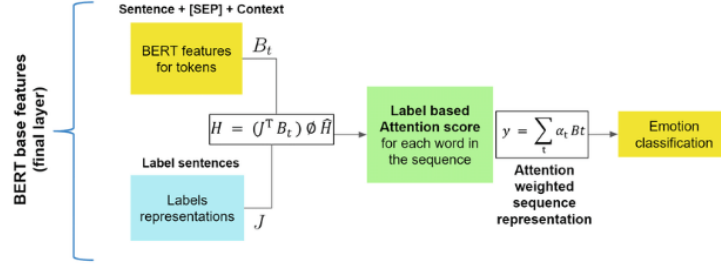


Figure 5.3: BERT for emotion classification

Source: [kaggle \[14\]](#)

BERT makes use of Transformer, an attention mechanism that learns contextual relations between words (or sub-words) in a text. In its vanilla form, Transformer includes two separate mechanisms — an encoder that reads the text input and a decoder that produces a prediction for the task. Since BERT’s goal is to generate a language model, only the encoder mechanism is necessary. The detailed workings of Transformer are described in a paper by Google.

As opposed to directional models, which read the text input sequentially (left-to-right or right-to-left), the Transformer encoder reads the entire sequence of words at once. Therefore it is considered bidirectional, though it would be more accurate to say that it’s non-directional. This characteristic allows the model to learn the context of a word based on all of its surroundings (left and right of the word).

5.2.2.2 Adam Optimizer

Adaptive Moment Estimation is an algorithm for optimization technique for gradient descent. The method is really efficient when working with large problem involving a lot of data or parameters. It requires less memory and is efficient. Intuitively, it is a combination of the ‘gradient descent with momentum’ algorithm and the ‘RMSP’ algorithm. Adam optimizer involves a combination of two gradient descent methodologies:

- **Momentum:** This algorithm is used to accelerate the gradient descent algorithm by taking into consideration the ‘exponentially weighted average’ of the gradients. Using averages makes the algorithm converge towards the minima in a faster pace.

$$w_{t+1} = w_t - \alpha m_t$$

$$m_t = \beta m_{t-1} + (1 - \beta) \left[\frac{\delta L}{\delta w_t} \right]$$

where:

- m_t : Current momentum at time t .

- m_{t-1} : Previous momentum at time $t - 1$.
 - w_t : Current weight at time t .
 - w_{t+1} : Updated weight at time $t + 1$.
 - α : Learning rate.
 - δ_L : Partial derivative of the loss L with respect to the weight w_t .
 - δw_t : Change in weight at time t .
 - β : Momentum decay parameter.
- Root Mean Square Propagation (RMSP): Root mean square prop or RM-Sprop is an adaptive learning algorithm that tries to improve AdaGrad. Instead of taking the cumulative sum of squared gradients like in AdaGrad, it takes the ‘exponential moving average’.

$$w_{t+1} = w_t - \frac{\alpha_t}{\sqrt{v_t + \epsilon}} \cdot \frac{\delta L}{\delta w_t}$$

$$v_t = \beta v_{t-1} + (1 - \beta) \left(\frac{\delta L}{\delta w_t} \right)^2$$

where:

- w_t : Weights at time t .
- w_{t+1} : Weights at time $t + 1$.
- α_t : Learning rate at time t .
- δ_L : Derivative of the loss function
- δw_t : derivative of weights at time t
- v_t : Sum of the square of past gradients, initially set to 0.
- β : Moving average parameter (constant, typically 0.9).
- ϵ : A small positive constant (e.g., 10^{-8}).

Adam Optimizer inherits the strengths or the positive attributes of the above two methods and builds upon them to give a more optimized gradient descent.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{\delta L}{\delta w_t}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(\frac{\delta L}{\delta w_t} \right)^2$$

Parameters Used:

- ϵ : A small positive constant to avoid ‘division by 0’ error when $v_t \rightarrow 0$ (e.g., 10^{-8}).
- β_1 and β_2 : Decay rates of the averages of gradients (typically, $\beta_1 = 0.9$ and $\beta_2 = 0.999$).
- α : Step size parameter or learning rate (e.g., 0.001).

5.2.2.3 Categorical Cross Entropy

Categorical Cross Entropy is also known as Softmax Loss. It's a softmax activation plus a Cross-Entropy loss used for multiclass classification. Using this loss, we can train a Convolutional Neural Network to output a probability over the N classes for each image.

In multiclass classification, the raw outputs of the neural network are passed through the softmax activation, which then outputs a vector of predicted probabilities over the input classes.

In the specific (and usual) case of multi-class classification, the labels are one-hot, so only the positive class keeps its term in the loss. There is only one element of the target vector, different than zero. Discarding the elements of the summation which are zero due to target labels, we can write:

$$CE = -\log \left(\frac{e^s p}{\sum_{j=1}^c e^{s_j}} \right)$$

5.2.3 Intents Recognition

In order to determine intents from text, we have extracted features from text using tf-idf vectorizer and then passed the input to various models such as: Support Vector Machine (SVM), Adaboost and Gradient boosting.

5.2.3.1 TF-IDF

TF-IDF stands for term frequency-inverse document frequency and it is a measure, used in the fields of information retrieval (IR) and machine learning, that can quantify the importance or relevance of string representations (words, phrases, lemmas, etc) in a document amongst a collection of documents (also known as a corpus).

Term frequency works by looking at the frequency of a particular term you are concerned with relative to the document. There are multiple measures, or ways, of defining frequency:

- Number of times the word appears in a document (raw count).
- Term frequency adjusted for the length of the document (raw count of occurrences divided by number of words in the document).
- Logarithmically scaled frequency (e.g. $\log(1 + \text{raw count})$).
- Boolean frequency (e.g. 1 if the term occurs, or 0 if the term does not occur, in the document).

Inverse document frequency looks at how common (or uncommon) a word is amongst the corpus. IDF is calculated as follows where t is the term (word) we are looking to measure the commonness of and N is the number of documents

(d) in the corpus (D). The denominator is simply the number of documents in which the term, t , appears in.

$$idf(t, D) = \log(N / (\text{count}(d \in D : t \in d)))$$

The reason we need IDF is to help correct for words like “of”, “as”, “the”, etc. since they appear frequently in an English corpus. Thus by taking inverse document frequency, we can minimize the weighting of frequent terms while making infrequent terms have a higher impact. Finally IDFs can also be pulled from either a background corpus, which corrects for sampling bias, or the dataset being used in the experiment at hand.

To summarize the key intuition motivating TF-IDF is the importance of a term is inversely related to its frequency across documents. TF gives us information on how often a term appears in a document and IDF gives us information about the relative rarity of a term in the collection of documents. By multiplying these values together we can get our final TF-IDF value. The higher the TF-IDF score the more important or relevant the term is; as a term gets less relevant, its TF-IDF score will approach 0.

$$tfidf(t, d, D) = tf(t, d).idf(t, D)$$

5.2.3.2 Support Vector Machines

Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning. The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n -dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane.

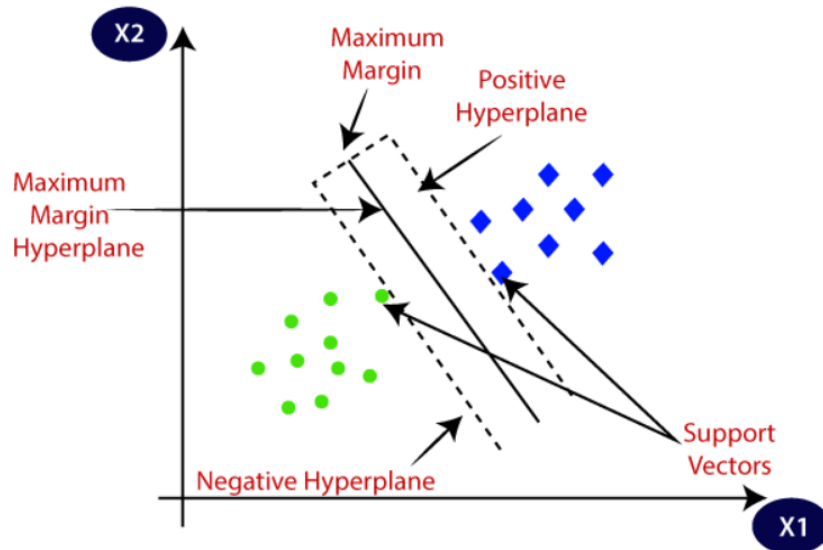


Figure 5.4: Support Vector Machine
Source: Research Gate [15]

5.2.3.3 AdaBoost

It is a part of the advanced ensemble learning models family and follows the way that boosting learning algorithm. Talking about the main idea behind the AdaBoost algorithm, it iteratively trains a sequence of weak classifiers on different subsets of the training data. During each iteration, the algorithm assigns higher weights to the misclassified samples from the previous iteration, thereby focusing on the more challenging examples. This process allows the subsequent weak classifiers to pay more attention to the previously misclassified samples and improve their performance.

The working of the AdaBoost algorithm in step by step manner is mentioned below:

1. When the algorithm is given data, it starts by Assigning equal weights to all training examples in the dataset. These weights represent the importance of each sample during the training process.
2. Here, this algorithm iterates with a few algorithms for a specified number of iterations (or until a stopping criterion is met). The algorithm trains a weak classifier on the training data. Here the weak classifier can be considered a model that performs slightly better than random guessing, such as a decision stump (a one-level decision tree).
3. During each iteration, the algorithm trains the weak classifier on given training data with the current sample weights. The weak classifier aims to minimize the classification error, weighted by the sample weights.

4. After training the weak classifier, the algorithm calculates classifier weight based on the errors of the weak classifier. A weak classifier with a lower error receives a higher weight.
5. Once the calculation of weight completes, the algorithm updates sample weights, and the algorithm gives assigns higher weights to misclassified examples so that more importance in subsequent iterations can be given.
6. After updating the sample weights, they are normalized so that they sum up to 1 and Combine the predictions of all weak classifiers using a weighted majority vote. The weights of the weak classifiers are considered when making the final prediction.
7. Finally, Steps 2–5 are repeated for the specified number of iterations (or until the stopping criterion is met), with the sample weights updated at each iteration. The final prediction is obtained by aggregating the predictions of all weak classifiers based on their weights.

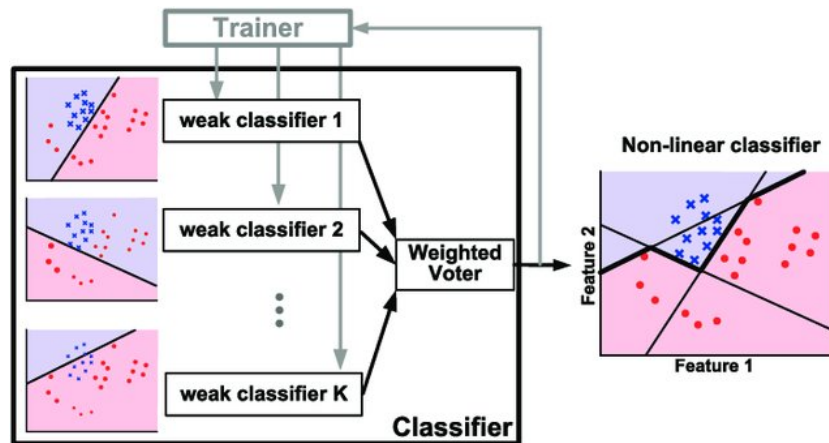


Figure 5.5: AdaBoost Classifier
Source: [Research Gate \[16\]](#)

5.2.4 Gradient Boosting

Gradient boosting is a machine learning technique used for both regression and classification problems. It builds a predictive model in the form of an ensemble of weak prediction models, typically decision trees, in a sequential manner. The key idea behind gradient boosting is to combine multiple weak learners, typically decision trees, to create a strong learner. It works as follows:

1. **Sequential Model Building:** The algorithm starts by initializing the model with a simple prediction, often the mean value for regression or the most frequent class for classification.
2. **Building Trees:** It then builds decision trees iteratively to correct the errors made by the existing model. Each new tree is built to predict the residuals

(the differences between the observed and predicted values) of the previous model.

3. **Gradient Descent Optimization:** It uses gradient descent optimization to minimize a loss function (like mean squared error for regression or cross-entropy loss for classification) when adding new trees. The gradient provides the direction to adjust the parameters of the model in order to minimize the loss.
4. **Adding Trees:** Additional trees are added one at a time, and each tree is trained on the residuals left by the previous trees. This process continues until a predefined number of trees is built, or until a certain level of performance is reached.
5. **Combining Predictions:** Finally, the predictions from all the trees are combined to make the final prediction. In regression problems, the final prediction is usually the sum of the predictions from all the trees. In classification problems, the final prediction is often obtained by taking a majority vote or averaging the predicted probabilities.

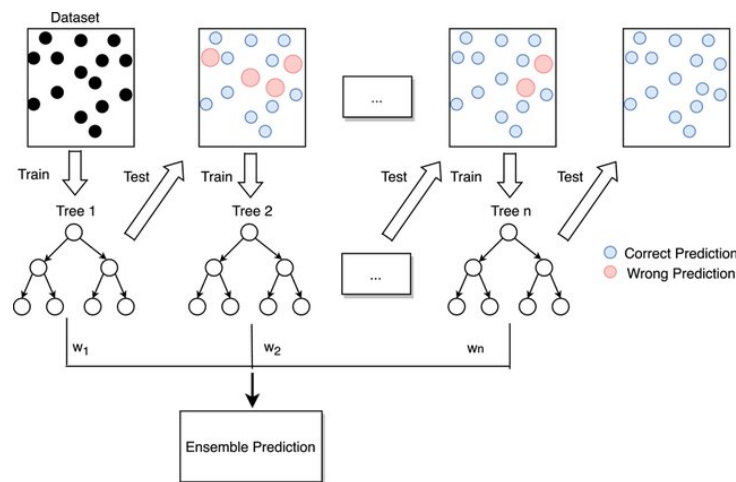


Figure 5.6: Gradient Boosting

Source: Research Gate [17]

5.2.5 Language Model

5.2.5.1 Transformer

A transformer architecture consists of an encoder and decoder that work together. The attention mechanism lets transformers encode the meaning of words based on the estimated importance of other words or tokens. This enables transformers to process all words or tokens in parallel for faster performance, helping drive the growth of increasingly bigger LLMs.

Thanks to the attention mechanism, the encoder block transforms each word or token into vectors further weighted by other words.

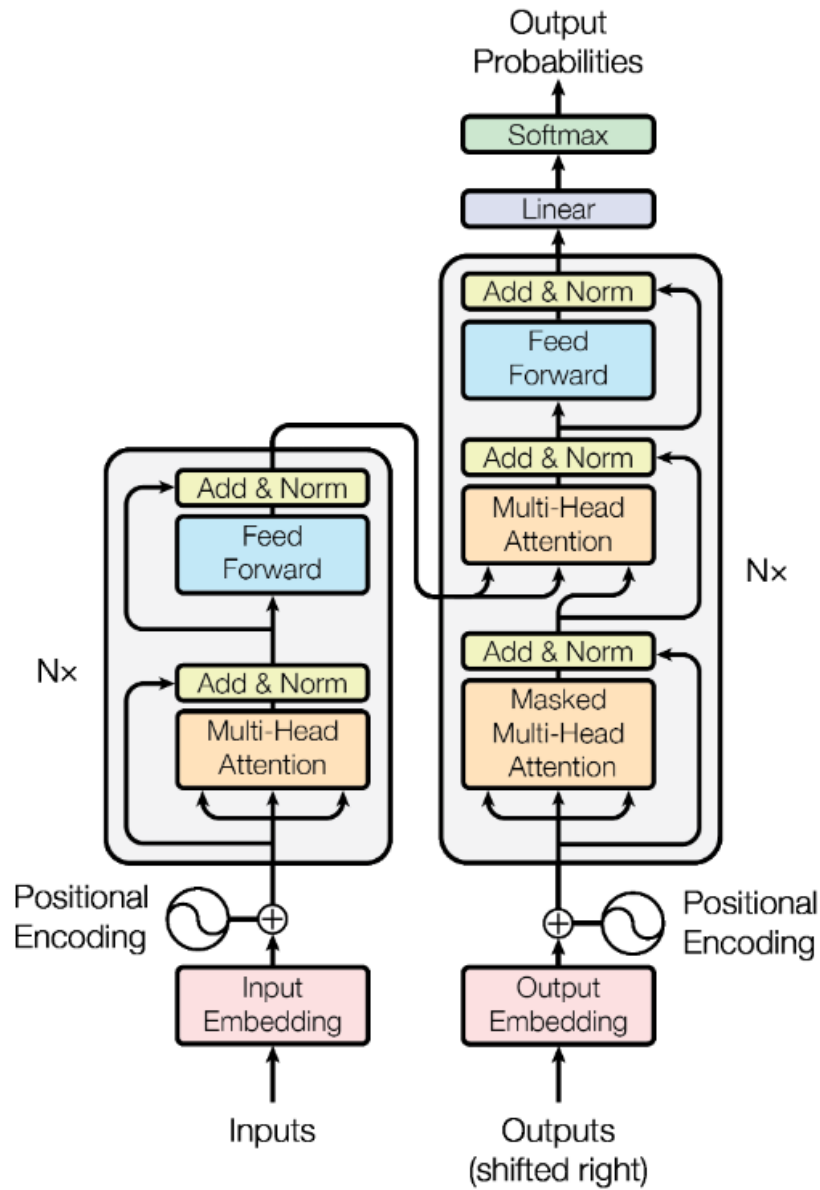


Figure 5.7: Transformer
Source: [Attention Is All You Need \[18\]](#)

The Transformer Attention The main components used by the Transformer attention are the following:

- q and k denoting vectors of dimension, d_k , containing the queries and keys, respectively
- v denoting a vector of dimension, d_v , containing the values

- Q , K , and V denoting matrices packing together sets of queries, keys, and values, respectively.
- W^Q , W^K and W^V denoting projection matrices that are used in generating different subspace representations of the query, key, and value matrices
- W^O denoting a projection matrix for the multi-head output

In essence, the attention function can be considered a mapping between a query and a set of key-value pairs to an output.

The Transformer implements a scaled dot-product attention, which follows the procedure of the general attention mechanism. The scaled dot-product attention first computes a dot product for each query, q , with all of the keys, k . It subsequently divides each result by $\sqrt{d_k}$ and proceeds to apply a softmax function. In doing so, it obtains the weights that are used to scale the values, v .

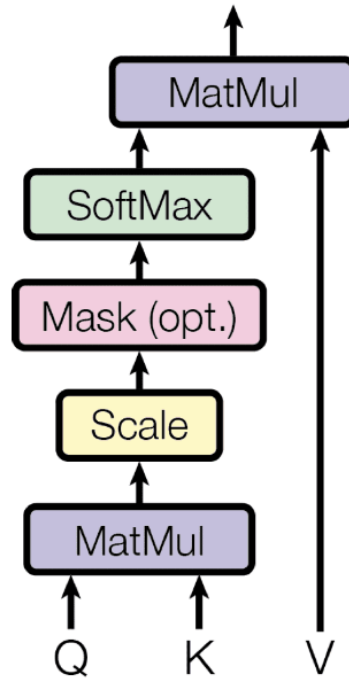


Figure 5.8: Scaled dot product attention
Source: [Machine Learning Mastery \[19\]](#)

In practice, the computations performed by the scaled dot-product attention can be efficiently applied to the entire set of queries simultaneously. In order to do so, the matrices — Q , K , and V —are supplied as inputs to the attention function:

$$\text{attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (5.1)$$

Feed-Forward Network In a transformer architecture, the feed-forward layer is an essential component of the model. It is a type of neural network layer that

is applied independently to each position in the sequence of inputs. The feed-forward layer consists of two linear transformations with a non-linear activation function in between them. The feed-forward layer is typically used multiple times in a transformer architecture, allowing the model to capture different patterns and dependencies in the data. It plays a crucial role in the overall functioning of the transformer model, contributing to the model's ability to learn and represent complex relationships in the input sequence.

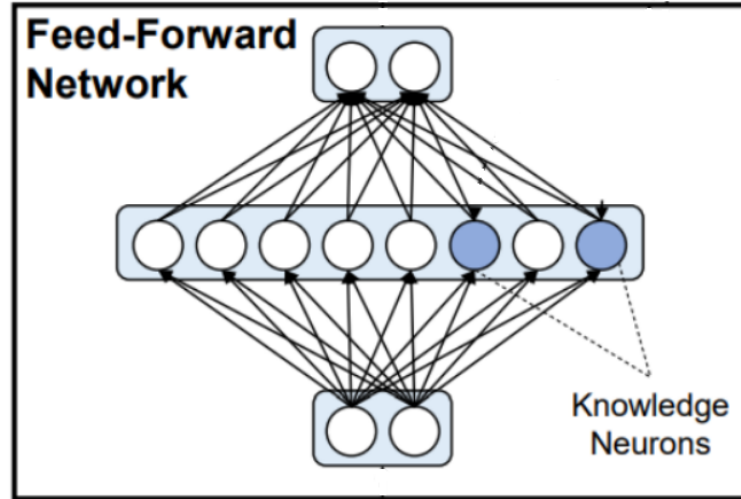


Figure 5.9: Feed-Forward Network

Source: Synced [20]

Softmax Layer The softmax layer in a Transformer model is a component used for generating probability distributions over a set of discrete elements. It is typically applied to the output of the final layer in the decoder part of the Transformer. In the context of language generation tasks, the Transformer's decoder generates a sequence of hidden states representing the predicted tokens. Each hidden state corresponds to a particular position in the output sequence. The softmax layer is then applied to these hidden states to convert them into a probability distribution over the vocabulary.

The softmax function takes a vector of real-valued scores (logits) and normalizes them into a probability distribution. It exponentiates each element of the input vector and divides it by the sum of the exponentiated elements, ensuring that the resulting probabilities sum up to 1.

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}} . \quad (5.2)$$

5.2.5.2 Sequence to Sequence Model

Recurrent Neural Network based Sequence-to-sequence (Seq2Seq) model is one of the most commonly researched model to implement artificial intelligence chat-bot. It is a model based on a recurrent neural network (RNN) that reads a word

from an input sentence one by one and then predicts the output word, which is concatenated to be a sentence. Basic seq2seq architecture structured of two recurrent neural networks (RNNs), one as an encoder processing the input and one as a decoder generating the response. However, there has been no experiments and analysis conducted to understand how Seq2Seq model handles variations in questions posed to it. There are three key components in the model:

1. Embedding - Embedding can be of type word or character or other forms such as bigram, trigram or even a hybrid between them. The function of the embedding layer is to convert the input into a vector of real numbers that represents the input. There are 2 options that we had to do that, by using one hot encoded bag of words (bow) or word2vec (CBOW).

Word2vec is a technique for natural language processing published in 2013. The word2vec algorithm uses a neural network model to learn word associations from a large corpus of text. Once trained, such a model can detect synonymous words or suggest additional words for a partial sentence. The basic idea is that the model creates word vectors by looking at the context with which words appear in sentences. Words with similar contexts will be placed close together in the vector space.

2. Encoder – The encoder RNN conceives a sequence of context tokens one at a time and updates its hidden state. After processing the whole context sequence, it produces a final hidden state, which incorporates the sense of context and is used for generating the answer.
3. Decoder – The goal of the decoder is to take context representation from the encoder and generate an answer. For this purpose, a softmax layer over vocabulary is maintained in the decoder RNN. At each time step, this layer takes the decoder hidden state and outputs a probability distribution over all words in its vocabulary.

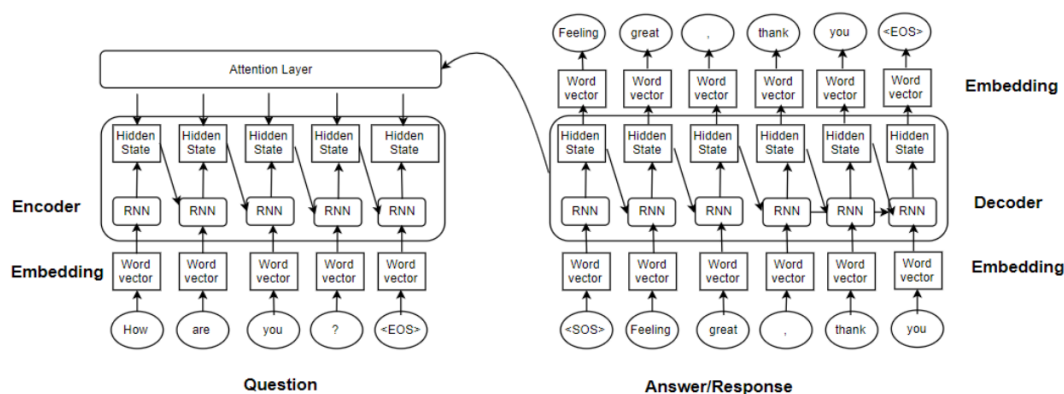


Figure 5.10: Illustration of a Seq2Seq Model with Attention

Although word embedding was originally used for Seq2Seq model, it poses a serious limitation if large dataset is used. The vocabulary will be very large and

requires huge processing power and long time to train. In order to address it researchers limit the vocabulary size. However, that itself poses another limitation whereby rarely occurring words will be left out from the vocabulary thus the model learning and may impact model performance especially question-answering system. One method to address this limitation is to use character embedding or sub-word embedding. An RNN also has problem of vanishing-gradient. Long short term memory (LSTM) or gated recurrent unit (GRU) are the most dominant variant of RNNs which used to learn the conversational dataset in these models. Also, the attention mechanism will allow the decoder to focus only on some important parts in the input at every decoding step.

5.3 Evaluation Metrics

Evaluation metrics play a crucial role in assessing the performance of machine learning models across various tasks, including classification, and language modeling. These metrics provide quantitative measures that help in understanding how well a model is performing and whether it meets the desired objectives. Here, we'll discuss some common evaluation metrics such as accuracy, precision, recall, F1-score, accuracy curve, and perplexity.

5.3.1 Accuracy

The most straightforward way to measure a classifier's performance is using the Accuracy metric. Here, we compare the actual and predicted class of each data point, and each match counts for one correct prediction. Accuracy is then given as the number of correct predictions divided by the total number of predictions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where:

- TP is the number of instances that are actually positive and are correctly identified as positive by the model.
- FP is the number of instances that are actually negative but are incorrectly identified as positive by the model.
- TN is the number of instances that are actually negative and are correctly identified as negative by the model.
- FN is the number of instances that are actually positive but are incorrectly identified as negative by the model.

5.3.2 Precision

Precision is a measure of a model's performance that tells us how many of the positive predictions made by the model are actually correct. It is calculated as

the number of true positive predictions divided by the number of true positive and false positive predictions.

$$Precision = \frac{TP}{TP + FP}$$

5.3.3 Recall

Recall measures the ability of a model to correctly identify all relevant instances within a dataset. Mathematically, recall is defined as the ratio of true positives to the sum of true positives and false negatives.

$$Recall = \frac{TP}{TP + FN}$$

5.3.4 F1-Score

The F1 score combines the precision and recall of a model into a single value. It provides a balance between these two metrics and is especially useful when the classes are imbalanced.

$$F1 - Score = \frac{2 * Precision * Recall}{Precision + Recall} = \frac{2 * TP}{2 * TP + FP + FN}$$

5.3.5 Learning Curve

This curve plots the performance (usually accuracy) of a model on the training set and the validation set against the number of training samples or iterations. It helps in understanding how the model's performance evolves as more data is provided or as training progresses. Typically, the accuracy increases as the model is exposed to more data, but it might stabilize eventually.

5.3.6 Perplexity

Perplexity is a concept commonly used in natural language processing (NLP), particularly in the context of evaluating language models, especially those based on probabilistic approaches. Perplexity measures how well a probability distribution or a probabilistic model predicts a sample. In the context of language modeling, perplexity quantifies how well a language model predicts a given sequence of words. A lower perplexity indicates that the model is better at predicting the sample, while a higher perplexity suggests poorer performance. Mathematically, perplexity is defined as the exponentiated average negative log-likelihood per word.

$$Perplexity = 2^{-\frac{1}{N} \sum_{i=1}^N \log_2 P(w_i)}$$

Where:

- N is the total number of words in the text.
- $P(w_i)$ is the probability assigned to word w_i by the language model.

5.4 Model Training

5.4.1 Voice Emotion Recognition Model

5.4.1.1 Data Collection

For voice emotion recognition, we have used RAVDESS Emotional speech audio dataset from kaggle [21]. The RAVDESS contains 24 professional actors (12 female, 12 male), vocalizing two lexically-matched statements in a neutral North American accent. Speech emotions includes calm, happy, sad, angry, fearful, surprise, and disgust expressions. For our purpose, we have utilized a balanced 400 audio samples from every emotions.

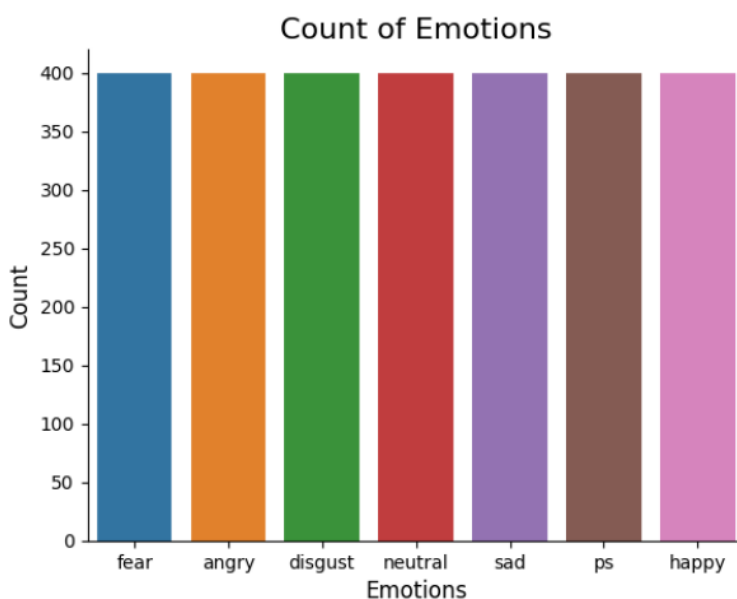


Figure 5.11: Emotion Counts

5.4.1.2 Visualization of Audio files

We have used waveplot and spectrogram in order to visualize audio files with different emotions.

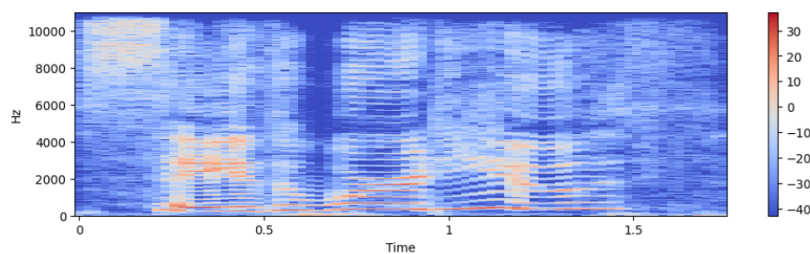


Figure 5.12: Spectrogram for Audio with Fear Emotion

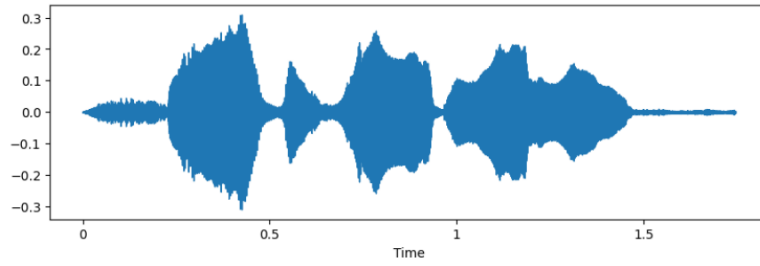


Figure 5.13: Waveplot for Audio with Fear Emotion

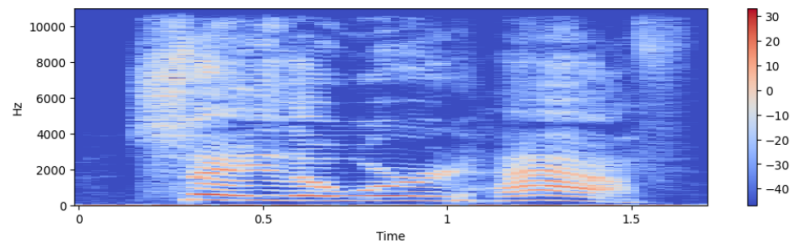


Figure 5.14: Spectrogram for Audio with Angry Emotion

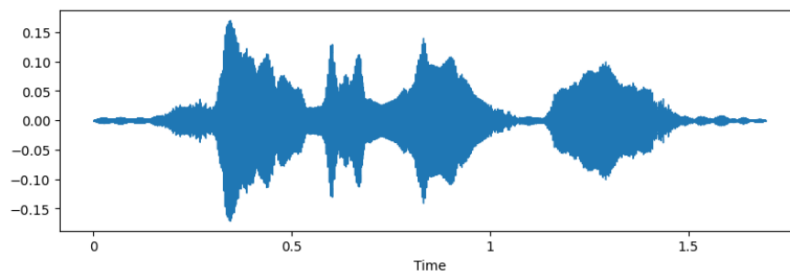


Figure 5.15: Waveplot for Audio with Angry Emotion

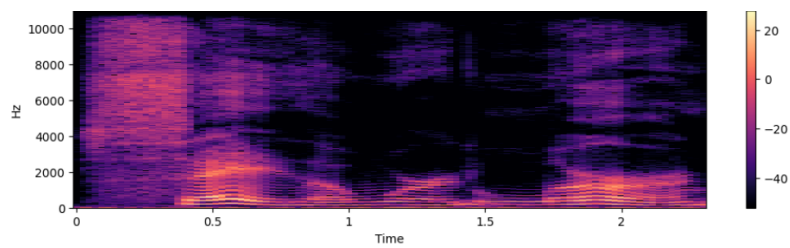


Figure 5.16: Spectrogram for Audio with Disgust Emotion

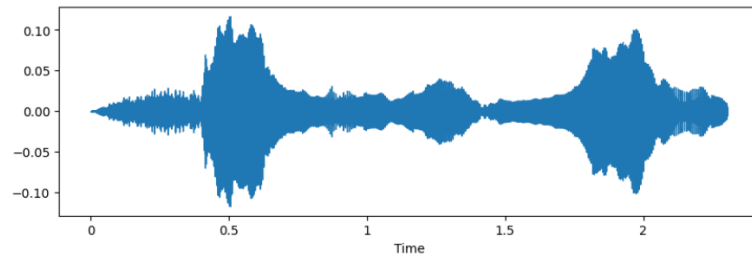


Figure 5.17: Waveplot for Audio with Disgust Emotion

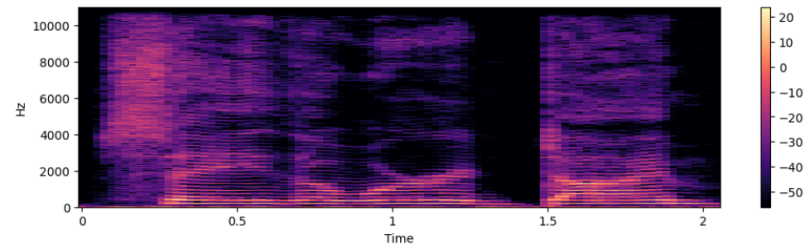


Figure 5.18: Spectrogram for Audio with Neutral Emotion

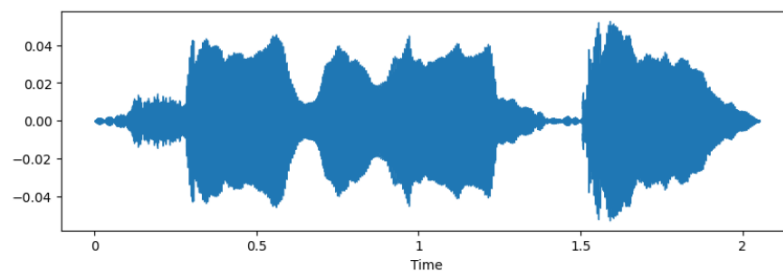


Figure 5.19: Waveplot for Audio with Neutral Emotion

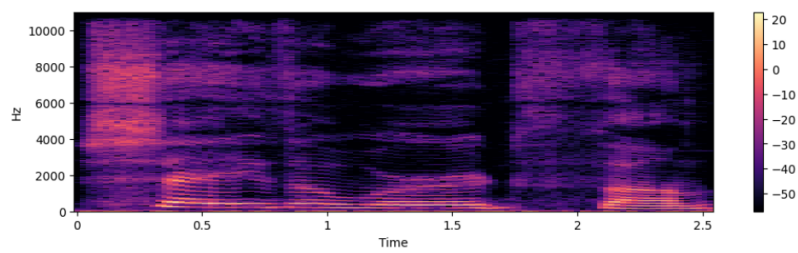


Figure 5.20: Spectrogram for Audio with Sad Emotion

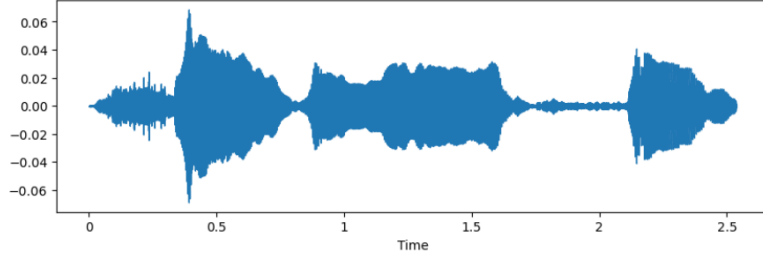


Figure 5.21: Waveplot for Audio with Sad Emotion

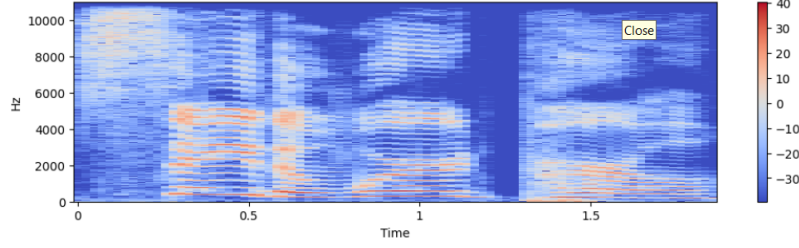


Figure 5.22: Spectrogram for Audio with Happy Emotion

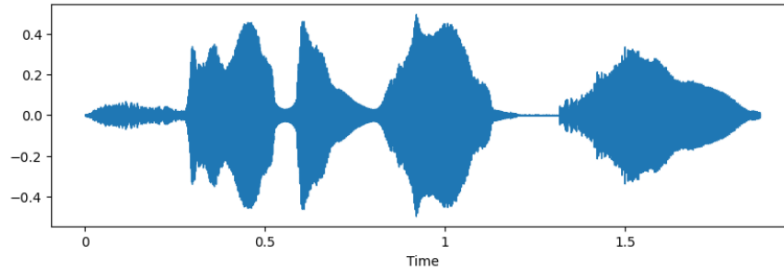


Figure 5.23: Waveplot for Audio with Happy Emotion

5.4.1.3 1d-CNN

After the feature extraction from audio samples and one-hot encoding of the label field, we used a 1D CNN architecture in order to process the input and classify the emotions. Our Voice emotion recognition architecture consists of following components:

1. Input Layer: The input shape is defined by `input_shape=(x_train.shape[1], 1)`, indicating the input data shape for the first layer. It's a 1D convolutional layer, expecting input data with a shape (sequence_length, 1). The 'same' padding ensures that the output has the same length as the input.
2. Convolutional Layers: There are four convolutional layers:
 - The first convolutional layer ('Conv1D') has 256 filters with a kernel size of 5 and ReLU activation function.

- Subsequent convolutional layers follow a similar pattern, gradually reducing the number of filters (256, 128, 64) but keeping the kernel size, padding, and activation function the same.
3. Pooling Layers: After each convolutional layer, there is a max-pooling layer ('MaxPooling1D') with a pool size of 5 and a stride of 2. This operation reduces the spatial dimensions of the input, aiding in extracting dominant features and reducing computation.
 4. Dropout Layers: Dropout layers ('Dropout') with a dropout rate of 0.2 or 0.3 are added after some convolutional layers and the first fully connected layer. Dropout is a regularization technique used to prevent overfitting by randomly setting a fraction of input units to zero during training.
 5. Flatten Layer: After the last convolutional layer, a flatten layer ('Flatten') is added to convert the 3D tensor output from the convolutional layers into a 1D tensor. This prepares the data for input into the fully connected layers.
 6. Fully Connected Layers:
 - The first dense layer ('Dense') has 32 units with ReLU activation and dropout of 0.3.
 - The output layer ('Dense') has 7 units corresponding to the number of emotions, with softmax activation function to output class probabilities.

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 20, 256)	1536
max_pooling1d (MaxPooling1D)	(None, 10, 256)	0
conv1d_1 (Conv1D)	(None, 10, 256)	327936
max_pooling1d_1 (MaxPooling1D)	(None, 5, 256)	0
dropout (Dropout)	(None, 5, 256)	0
conv1d_2 (Conv1D)	(None, 5, 128)	163968
max_pooling1d_2 (MaxPooling1D)	(None, 3, 128)	0
dropout_1 (Dropout)	(None, 3, 128)	0
conv1d_3 (Conv1D)	(None, 3, 64)	41024
...		
Total params: 538,823		
Trainable params: 538,823		
Non-trainable params: 0		

Figure 5.24: Model Summary for Voice Emotion Recognition

5.4.2 Text Emotion Recognition Model

5.4.2.1 Data Collection

For this model we collected data from [22] which consisted of data from twitter with six basic emotions: anger,fear,joy,love,sadness and surprise. There are different annotations that we used for different emotions which are listed as below.

- Anger:0
- Fear:1
- Joy:2
- Love:3
- Sadness:4
- Surprise:5

Within the dataset we had two fields text and label. In this case text is a string feature and label is a classification feature with a possible value of 0,1,2,3,4,5 with the emotions designated as above.

A sample of the dataset has been shown in the figure below.

i didnt feel humiliated	0 sadness
i can go from feeling so hopeless to so damned hopeful just from being around someone who cares and is awake	0 sadness
im grabbing a minute to post i feel greedy wrong	3 anger
i am ever feeling nostalgic about the fireplace i will know that it is still on the property	2 love
i am feeling grouchy	3 anger
ive been feeling a little burdened lately wasnt sure why that was	0 sadness
ive been taking or milligrams or times recommended amount and ive fallen asleep a lot faster but i also feel like so funny	5 surprise

Figure 5.25: Text Emotion Model Data-Set Sample

The dataset has two configurations

- Split: With a total of 20,000 examples split into train,validation and split
- Unsplit: With a total of 4,16,809 examples in a single train split.

5.4.2.2 Text Preprocessing

Following where the steps carried out to perform data cleaning:

- Stopword Removal:Remove common words (stopwords) that do not contribute much meaning, such as "the," "and," "is," etc.
- Lemmatization: Lemmatization is to reduce a word to its root form
- Text cleaning: The text was converted to lowercase to ensure consistency and any abbreviations and contractions were removed. of the feature space.

5.4.2.3 Model Loading

For loading the model we carried out the following tasks:

- First we initialized a BERT tokenizer ("AutoTokenizer") and a BERT model ("TFBertModel") from the pre-trained bert-base-cased checkpoint
- Some of the layers related to masked language modeling (MLM) and next sentence prediction (NSP) were not used for our purpose
- The the tokenizer and BERT models were saved locally for later use
- A sample input "I will be a kaggle grandmaster" was tokenized producing a dictionary with 'input_ids', 'token_type_ids', and 'attention_mask'.
- Then we tokenized the training and testing data thus preparing it for input to BERT model.
- Then we construct a neural network for text classification utilizing BERT embeddings. Input layers are defined to accommodate tokenized sequences and attention masks.
- BERT embeddings are obtained by applying the pre-trained BERT model to the input data, followed by potential additional layers designed for the specific classification task. The model is configured with trainable BERT layers.
- Adam optimizer is employed with specified hyperparameters. Categorical crossentropy serves as the loss function, and the evaluation metric chosen is categorical accuracy with balanced accuracy. The model is compiled, aligning it for training on text classification tasks leveraging the power of BERT representations.
- The model summary is as shown below:

```
Model: "model"
```

Layer (type)	Output Shape	Param #	Connected to
input_ids (InputLayer)	[(None, 70)]	0	
attention_mask (InputLayer)	[(None, 70)]	0	
tf_bert_model (TFBertModel)	TFBaseModelOutputWit	108310272	input_ids[0][0] attention_mask[0][0]
global_max_pooling1d (GlobalMax	(None, 768)	0	tf_bert_model[0][0]
dense (Dense)	(None, 128)	98432	global_max_pooling1d[0][0]
dropout_37 (Dropout)	(None, 128)	0	dense[0][0]
dense_1 (Dense)	(None, 32)	4128	dropout_37[0][0]
dense_2 (Dense)	(None, 6)	198	dense_1[0][0]

```

Total params: 108,413,030
Trainable params: 108,413,030
Non-trainable params: 0

```

Figure 5.26: Model Summary for Text Emotion Recognition Model

- The model is trained using the training data ("x_train" and "data_train") by updating the models weight based on the optimization algorithm (Adam optimizer) using categorical cross-entropy as the loss function.

5.4.3 Intent Recognition Model

5.4.3.1 Data Collection

The data set that we used contained a total of 1271 rows. A sample of this dataset is provided below:

Types	Complaints	Comment
Stress/Anxiety	How do I overcome my anxiety and depression?	NaN
Counseling Fundamentals	Is it normal to cry at therapy?	NaN
Stress/Anxiety	How do I talk to my girlfriend about my anxiety?	NaN
Relationship Dissolution	My fiancé's ex-husband shows up unannounced	NaN
Behavioral Change	How do I stop sneaking away from home at night?	NaN
Stress/Anxiety	Anxiety about the passage of time	NaN
Family Conflict	How do I cope with "never being good enough?"	NaN

Table 5.1: Sample DataSet

In our dataset the Complaints were classified into numerous types. The count of each type is listed below:

Category	Count
Relationships	186
Stress/Anxiety	97
Depression	91
Intimacy	80
Social Relationships	75
Family Conflict	72
Anger Management	70
Trauma/Grief/Loss	69
LGBTQ	55
Parenting	54
Marriage	50
Substance Abuse/Addiction	50
Sleep Improvement	50
Self-esteem	49
Workplace issues	49
Behavioral Change	42
Relationship Dissolution	37
Counseling Fundamentals	33

Table 5.2: Category Counts

5.4.3.2 Data Visualization

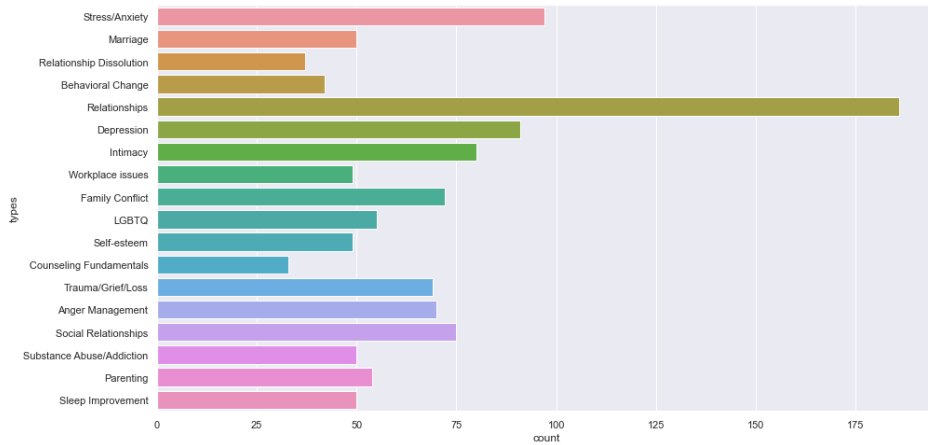


Figure 5.27: Data Visualization

5.4.3.3 Text Preprocessing

The various text preprocessing methods that were carried out were:

- Stop Words Removal
- Removing punctuations
- Tokenization
- Lemmatization

Then label encoding was carried out. The final data set looked like as follows:

Types	Complaints	Comment	Cleaned Message	Topic Encoding
Stress/Anxiety	How do I overcome my anxiety and depression?	NaN	overcome anxiety depression	14
Stress/Anxiety	How do I talk to my girlfriend about my anxiety?	NaN	talk girlfriend anxiety	14
Relationship Dissolution	My fiancé's ex-husband shows up unannounced	NaN	fiancé ex-husband show unannounced	9
Behavioral Change	How do I stop sneaking away from home at night?	NaN	stop sneak away home night	1
Stress/Anxiety	Anxiety about the passage of time	NaN	anxiety passage time	14
Workplace issues	What to do about a boss that cannot stop lying	NaN	boss stop lie	17

Table 5.3: Sample Data with Cleaned Messages and Topic Encoding

5.4.3.4 Feature Extraction

Feature extraction is carried out using the TF-IDF (Term Frequency-Inverse Document Frequency) method. The dataset is split into training and testing sets using the `train_test_split` function. The 'cleanmsg' column from the DataFrame (df) is used as the input data (x) for training and testing, and the 'topic_encoding' column is used as the target variable (y). The TF-IDF feature extraction process involves two main steps: Count Vectorization and TF-IDF Transformation.

First, a `CountVectorizer` is initialized with the option to consider n-grams within the specified range (in this case, unigrams, bigrams, and trigrams with `ngram_range=(1,3)`).

This vectorizer converts the text data into a matrix of token counts.

Then, a `TfidfTransformer` is applied to the counts obtained from the `CountVectorizer`. The TF-IDF transformation computes the Term Frequency-Inverse Document Frequency scores, which reflect the importance of each term within each document relative to its frequency across the entire dataset. The `sublinear_tf=True` option ensures that the term frequencies are replaced with their logarithmic form. The resulting TF-IDF matrices (`x_train_tfidf` and `x_test_tfidf`) now represent the input data with numerical features derived from the original text, incorporating information about the importance of each term in distinguishing between different topics.

5.4.3.5 Model Building

- Ensembling SVM with AdaBoost
 - We implemented an AdaBoost classifier using scikit-learn’s `AdaBoostClassifier` with an SGD (Stochastic Gradient Descent) classifier as the base estimator. The base estimator is configured with hinge loss, and the SAMME (Stagewise Additive Modeling using a Multiclass Exponential loss function) algorithm is employed. The learning rate is set to 0.5, and the number of weak learners (estimators) is specified as 300.
 - The AdaBoost model was trained on the TF-IDF transformed training data (`x_train_tfidf`) along with the corresponding labels (`y_train`).
 - We used the trained AdaBoost model to make predictions on the TF-IDF transformed test data (`x_test_tfidf`).
 - We assessed the model’s performance by calculating the accuracy score and generating a detailed classification report, which includes precision, recall, and F1-score which are reported below:
- Gradient boosting
 - We started by importing the required libraries, including scikit-learn’s `GradientBoostingClassifier`
 - A Gradient Boosting Classifier (`model_gbc`) was instantiated with specific hyperparameters. Notably, we configured settings such as:
 - * The number of estimators:1000
 - * Maximum features:’auto’
 - * Maximum depth: 4
 - * Random state : 1
 - * Verbosity for monitoring the training process.
 - The Gradient Boosting Classifier was trained using the TF-IDF transformed training data (`x_train_tfidf`) and corresponding labels (`y_train`). This process involved the iterative improvement of the model’s performance through boosting.

- With the trained model, we made predictions on the TF-IDF transformed test data (`x_test_tfidf`).
- Grid Searching With SVM
 - We specified a dictionary `param_grid` containing various values for
 - * The regularization parameter '`C`': [0.1, 1, 10, 100, 1000]
 - * Kernel coefficient '`gamma`': [1, 0.1, 0.01, 0.001, 0.0001]
 - * '`kernel`': ['rbf', 'linear'].
 - GridSearchCV was initialized for SVM, using the defined hyperparameter grid with
 - * refitting (`refit=True`)
 - * verbosity (`verbose=3`)
 - The grid search was executed by fitting GridSearchCV to training data (`x_train_tfidf`, `y_train`).
 - The best hyperparameters were identified after the grid search which came out to be:
 - * `C:1000`
 - * `gamma:0.001`
 - * `kernel:rbf`

5.4.4 Language Model

5.4.4.1 Data Collection

For the generative model, we have used empathetic dialogues dataset from hugging face [23]. The dataset is originally from the paper [24] but is free to use in the platform. There are total of 76673 samples in train set, 12030 samples in validation set and 10943 samples in test set. The dataset consisted of following fields:

- `conv_id`: a string feature
- `utterance_idx`: an int32 feature
- `context`: a string feature
- `prompt`: a string feature
- `speaker_idx`: an int32 feature
- `utterance`: a string feature
- `selfeval`: a string feature
- `tags`: a string feature

During feature selection step, we removed all the fields except context, utterance and conv_id. The dataset initially encompassed conversations reflecting a wide spectrum of 32 distinct emotions, including sentiments such as faithful, terrified, grateful, excited, annoyed, prepared, joyful, angry, sad, jealous and so on. To streamline our project, we reclassified these emotions into 6 fundamental categories: Angry, Neutral, Disgust, Fear, Happy and Sad using the emotion model that we previously built. Also the subsequent intention is added using previously built intention recognition model.

5.4.4.2 Seq2Seq

For the generative part in our chatbot, we first implemented a sequence-to-sequence model using Long Short-Term Memory (LSTM) networks with an encoder-decoder architecture.

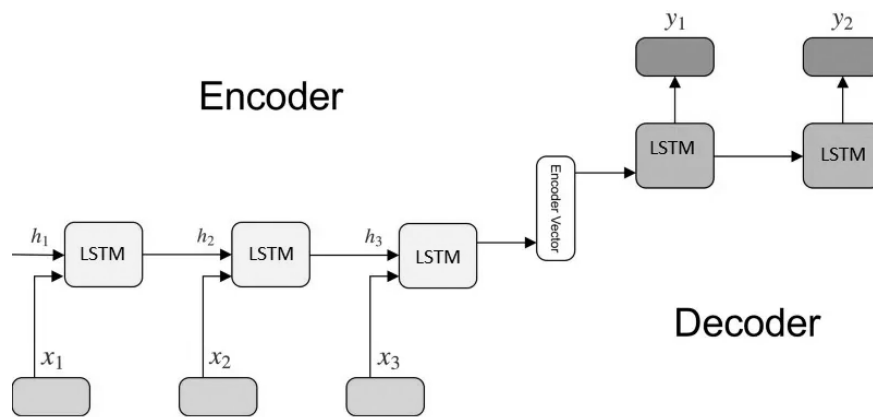


Figure 5.28: Block Diagram of seq2seq Architecture

1. Encoder
 - (a) input sequences are passed through an initial LSTM layer (LSTM1) with return sequences and states.
 - (b) The output sequences and states from LSTM1 are then fed into a second LSTM layer (LSTM2) with return sequences and states.
 - (c) The output sequences and states from LSTM2 are finally fed into a third LSTM layer (LSTM3) with return sequences and states.
2. Decoder
 - (a) Decoder input sequences are processed through an initial LSTM layer (LSTM1) with return sequences and states. The initial states are set to the final states obtained from the encoder.
 - (b) The output sequences from LSTM1 are then passed through a second LSTM layer (LSTM2) with return sequences and states.

- (c) The final output sequences from LSTM2 are fed into a dense layer with softmax activation, producing the model's output.

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, None, 86)]	0	[]
lstm (LSTM)	[(None, None, 256), (None, 256), (None, 256)]	351232	['input_1[0][0]']
input_2 (InputLayer)	[(None, None, 170)]	0	[]
lstm_1 (LSTM)	[(None, None, 256), (None, 256), (None, 256)]	525312	['lstm[0][0]']
lstm_2 (LSTM)	[(None, None, 256), (None, 256), (None, 256)]	437248	['input_2[0][0]', 'lstm_1[0][1]', 'lstm_1[0][2]']
lstm_3 (LSTM)	[(None, None, 256), (None, 256), (None, 256)]	525312	['lstm_2[0][0]', 'lstm_1[0][1]', 'lstm_1[0][2]']
dense (Dense)	(None, None, 170)	43690	['lstm_3[0][0]']

=====

Total params: 1882794 (7.18 MB)
Trainable params: 1882794 (7.18 MB)
Non-trainable params: 0 (0.00 Byte)

Figure 5.29: Model Summary for Seq2Seq Architecture

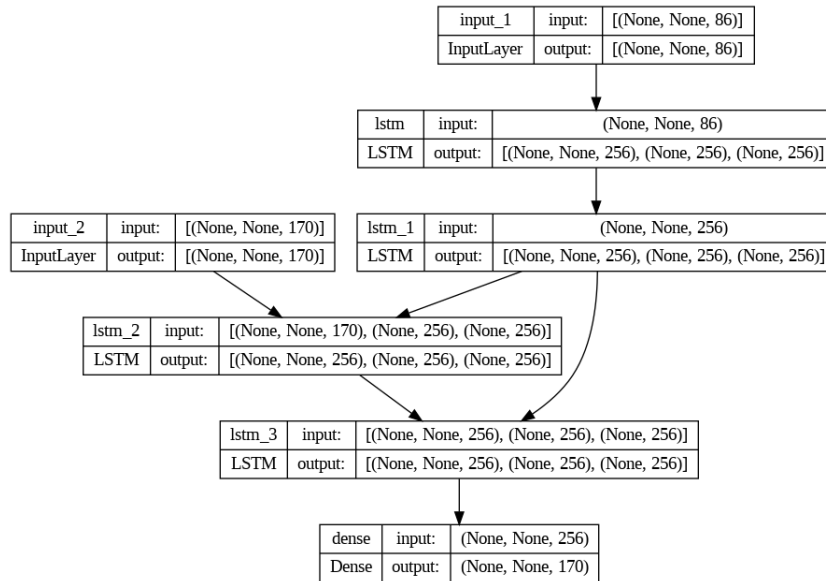


Figure 5.30: Seq2Seq Model Description

5.4.4.3 Transformer Architecture

Our custom transformer based architecture consists of following components:

1. Data Preprocessing:
 - The script loads data from a JSON file named 'empathetic.json', which presumably contains context, utterance and conv_id. for training our chatbot.

- Text data preprocessing is done using Tokenizer from TensorFlow's Keras preprocessing module. This step involves tokenizing the text data and converting it into sequences of integers.
- Both context and utterance are padded to a maximum length ('max len') to ensure uniformity in input dimensions.

2. Transformer Architecture:

- The core of the model lies in the Transformer architecture, which consists of self-attention mechanisms. This architecture allows the model to focus on different parts of the input sequence when generating an output, making it well-suited for sequence-to-sequence tasks like dialogue generation.

3. MultiHeadSelfAttention Layer:

- This layer computes the attention weights between different words in the input sequence. It divides the embedding vectors into multiple heads to capture different aspects of the input simultaneously.
- The attention mechanism helps the model to weigh the importance of each word in the input sequence concerning every other word, enabling it to understand the context better.

4. TransformerBlock Layer:

- This layer consists of multiple Transformer blocks stacked together. Each block contains a self-attention layer followed by a feed-forward neural network (FFN) layer.
- Layer normalization and dropout are applied within each block to stabilize the training process and prevent overfitting.

5. Encoder and Decoder:

- The model uses a dual-encoder-decoder architecture.
- The encoder takes input patterns and processes them to generate a fixed-length representation of the input sequence.
- The decoder takes input responses and generates another fixed-length representation.
- Both encoder and decoder consist of multiple Transformer blocks.

6. Concatenation and Output Layer:

- The outputs of the encoder and decoder are concatenated to capture the overall context of the conversation.
- A fully connected layer with softmax activation is used to predict the probability distribution over different intent labels.

7. Model Compilation and Training:

- The Adam optimizer is used for training, which is a popular choice for gradient-based optimization.
- The model is trained on the padded sequences of patterns and responses with their corresponding intent labels for a specified number of epochs.

8. Model Saving:

- After training, the trained model is saved, fitted tokenizer, and label encoder using pickle for future use in inference.

The model summary of our custom transformer architecture is shown below:

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, 20)]	0	[]
input_2 (InputLayer)	[(None, 20)]	0	[]
embedding (Embedding)	(None, 20, 32)	32000	['input_1[0][0]']
embedding_1 (Embedding)	(None, 20, 32)	32000	['input_2[0][0]']
transformer_block (TransformerBlock)	(None, 20, 32)	8544	['embedding[0][0]']
transformer_block_4 (TransformerBlock)	(None, 20, 32)	8544	['embedding_1[0][0]']
transformer_block_1 (TransformerBlock)	(None, 20, 32)	8544	['transformer_block[0][0]']
transformer_block_5 (TransformerBlock)	(None, 20, 32)	8544	['transformer_block_4[0][0]']
transformer_block_2 (TransformerBlock)	(None, 20, 32)	8544	['transformer_block_1[0][0]']
transformer_block_6 (TransformerBlock)	(None, 20, 32)	8544	['transformer_block_5[0][0]']
transformer_block_3 (TransformerBlock)	(None, 20, 32)	8544	['transformer_block_2[0][0]']
transformer_block_7 (TransformerBlock)	(None, 20, 32)	8544	['transformer_block_6[0][0]']
global_average_pooling1d (GlobalAveragePooling1D)	(None, 32)	0	['transformer_block_3[0][0]']
global_average_pooling1d_1 (GlobalAveragePooling1D)	(None, 32)	0	['transformer_block_7[0][0]']
concatenate (Concatenate)	(None, 64)	0	['global_average_pooling1d[0][0]', 'global_average_pooling1d_1[0][0]']
dense_48 (Dense)	(None, 129)	8385	['concatenate[0][0]']
Total params: 140,737 Trainable params: 140,737 Non-trainable params: 0			

Figure 5.31: Model Summary for Custom Transformer Architecture

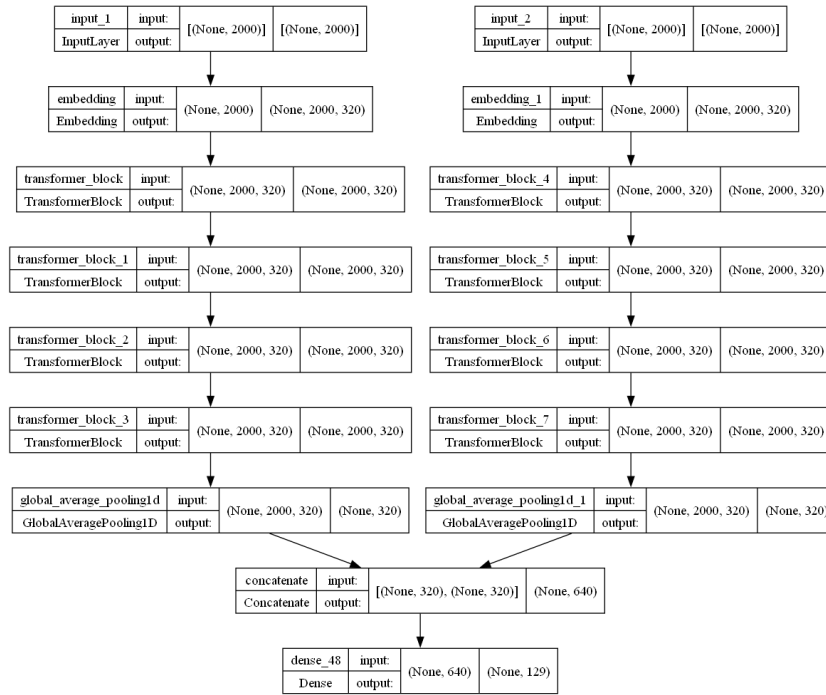


Figure 5.32: Custom Transformer Architecture

5.5 UI Development

The webapp was created using the following technologies:

- HTML
- CSS
- Javascript
- ExpressJS
- NodeJS
- FastAPI

Using the above mentioned technologies we built a easy to use and responsive web app with features to:

- Register for new user
- Sign in facilitates for already registered users

The basic layout of the website was written inside EJS files where EJS is a simple templating language that lets you generate HTML markup with plain JavaScript. Proper CSS stylings were added to the pages to beautify it and the backend routing between the pages was handled using ExpressJS.

Once the entire ensemble of models and user interfaces had been meticulously constructed and stored, they were seamlessly amalgamated through the prowess of FastAPI. In the preliminary stages, a trained voice recognition model was seamlessly integrated to capture and interpret user utterances. Subsequently, the processed speech underwent scrutiny by an emotion recognition model, adept at discerning the user's emotional state. Simultaneously, the spoken words underwent transformation into written text. The ensuing textual representation underwent a dual extraction process, disentangling both the emotion and intentions embedded within. The resultant amalgamation of emotion, textual content and user intent was then channeled into a transformer model. Within the intricate network of transformers, a judiciously crafted reply emerged, tailored to the user's input.

Chapter 6

Result and Discussion

6.1 Voice Emotion Detection Model

We splitted the data into 80-20 splits and passed the 20% of the data as validation set. The result of above voice emotion recognition model is shown as follows:

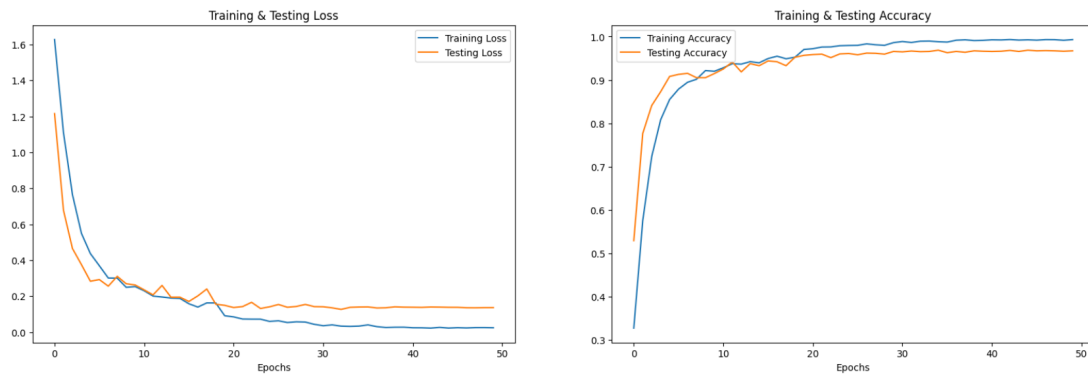


Figure 6.1: Accuracy and Loss Curve for Voice Emotion Recognition Model

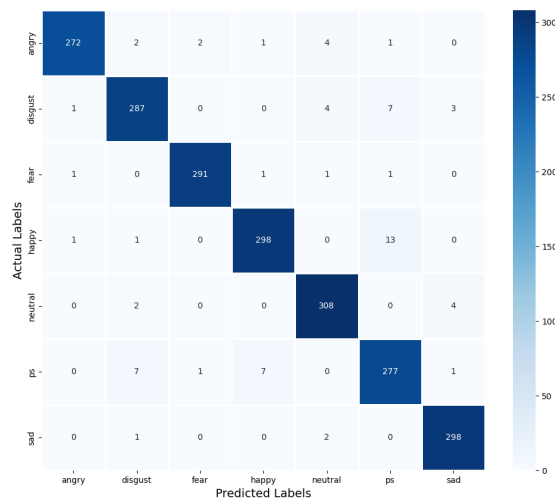


Figure 6.2: Confusion matrix for Voice Emotion Recognition Model

Class	Precision	Recall	F-score	Support
angry	0.99	0.96	0.98	282
disgust	0.96	0.95	0.95	302
fear	0.99	0.99	0.99	295
happy	0.97	0.95	0.96	313
neutral	0.97	0.98	0.97	314
ps	0.93	0.95	0.94	293
sad	0.97	0.99	0.98	301
accuracy			0.97	2100
macro avg	0.97	0.97	0.97	2100
weighted avg	0.97	0.97	0.97	2100

Table 6.1: Classification Report for Voice Emotion Recognition Model

Speech is a temporal signal, where the sequence of audio samples matters. A 1D CNN can effectively capture local temporal patterns in the input signal, which is crucial for understanding speech emotions. During the experiment, we have found that convolutional layers can learn features from smaller segments of the input, preserving temporal information due to which we were able to achieve accuracy as high as 97%.

6.2 Text Emotion Detection Model

The following table indicates the classification report for our text emotion detection model.

Class	Precision	Recall	F-score	Support
0	0.97	0.94	0.96	1739
1	0.95	0.94	0.95	2028
2	0.82	0.88	0.85	492
3	0.91	0.94	0.93	813
4	0.87	0.92	0.89	712
5	0.87	0.77	0.82	216
accuracy			0.93	6000
macro avg	0.90	0.90	0.90	6000
weighted avg	0.93	0.93	0.93	6000

Table 6.2: Classification Report for Text Emotion Recognition Model

The confusion matrix of text emotion assistance system is mentioned below:

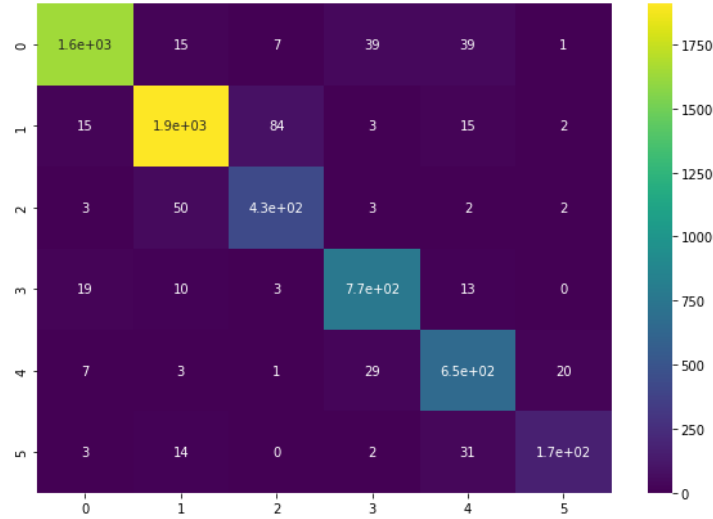


Figure 6.3: Confusion Matrix for Text Emotion Recognition Model

We found that, BERT captured contextual relationships between words in a sentence. It considered the entire input text sequence bidirectionally, allowing it to understand the context in which each word appears. This contextual understanding is crucial for accurately detecting emotions in text, as emotions are often conveyed through context. Also BERT is pretrained on large text corpora using unsupervised learning tasks such as masked language modeling and next sentence prediction. This pretrained model captures rich semantic representations of words and sentences, which can generalize well to various tasks, including emotion detection.

6.3 Intention Detection Model

Class	Precision	Recall	F-score	Support
0	0.88	0.41	0.56	17
1	0.75	0.21	0.33	14
2	1.00	0.20	0.33	10
3	0.48	0.31	0.37	36
4	0.29	0.19	0.23	21
5	0.86	0.25	0.39	24
6	1.00	0.35	0.52	17
7	0.80	0.29	0.42	14
8	1.00	0.22	0.36	18
9	0.04	0.44	0.08	9
10	0.57	0.58	0.57	67
11	0.08	0.53	0.14	19
12	1.00	0.38	0.55	16
13	0.38	0.18	0.24	28
14	0.76	0.33	0.46	40

15	1.00	0.42	0.59	12
16	1.00	0.18	0.31	22
17	1.00	0.07	0.12	15
accuracy			0.64	399
macro avg	0.72	0.31	0.37	399
weighted avg	0.68	0.34	0.40	399

Table 6.3: Classification Report for a Adaboost Classifier

Class	Precision	Recall	F-score	Support
0	0.76	0.76	0.76	17
1	0.57	0.29	0.38	14
2	0.89	0.80	0.84	10
3	0.45	0.39	0.42	36
4	0.36	0.43	0.39	21
5	0.47	0.38	0.42	24
6	0.62	0.59	0.61	17
7	0.35	0.43	0.39	14
8	0.53	0.56	0.54	18
9	0.29	0.22	0.25	9
10	0.42	0.69	0.52	67
11	0.26	0.26	0.26	19
12	0.92	0.69	0.79	16
13	0.44	0.57	0.50	28
14	0.65	0.42	0.52	40
15	1.00	0.42	0.59	12
16	0.86	0.55	0.67	22
17	0.91	0.67	0.77	15
accuracy			0.62	399
macro avg	0.60	0.51	0.53	399
weighted avg	0.56	0.52	0.52	399

Table 6.4: Classification Report for a Gradient Boosting

As we can see, both Adaboost and Gradient Boosting classifier didn't work well. So, we also tried SVM model with the optimal hyperparameters found using Grid-SearchCV as $C=1000$, $\gamma=0.001$ and $\text{kernel}=\text{rbf}$. Its classification report along with confusion matrix is shown below:

Class	Precision	Recall	F-score	Support
0	0.75	0.71	0.73	107
1	0.67	0.69	0.70	140
2	0.89	0.80	0.84	100
3	0.73	0.72	0.72	360
4	0.79	0.73	0.81	210
5	0.81	0.88	0.79	240
6	0.72	0.79	0.71	170
7	0.38	0.83	0.60	140
8	0.79	0.76	0.77	180
9	0.79	0.72	0.85	90
10	0.63	0.73	0.64	670
11	0.78	0.66	0.77	190
12	0.91	0.72	0.74	160
13	0.78	0.77	0.72	280
14	0.72	0.80	0.86	400
15	0.83	0.72	0.66	120
16	1.00	0.76	0.73	220
17	0.89	0.83	0.77	150
accuracy			0.72	3990
macro avg	0.70	0.79	0.76	3990
weighted avg	0.77	0.72	0.57	3990

Table 6.5: Classification Report for SVM

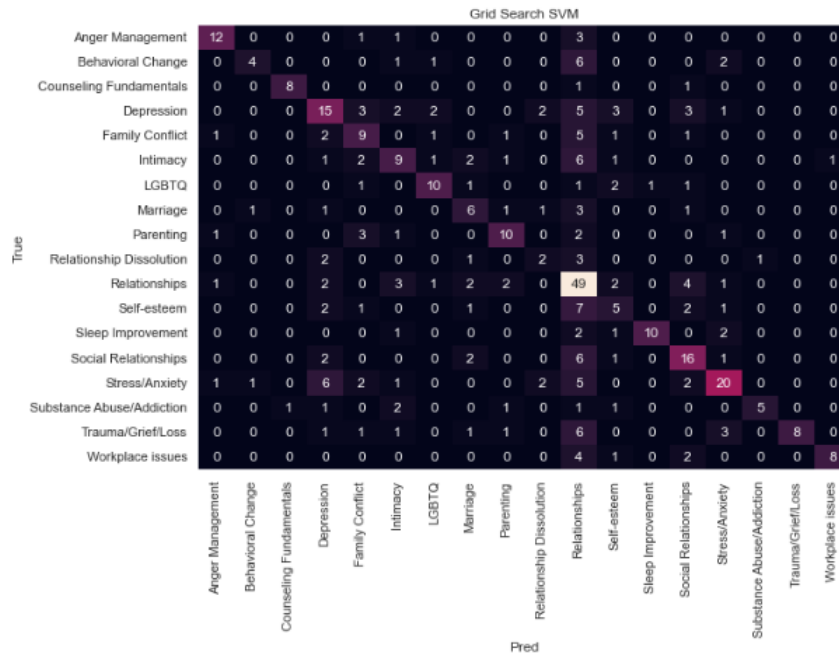


Figure 6.4: Confusion matrix for SVM

So an SVM model worked decent for intention recognition, achieving accuracy of 72%.

6.4 Language Model

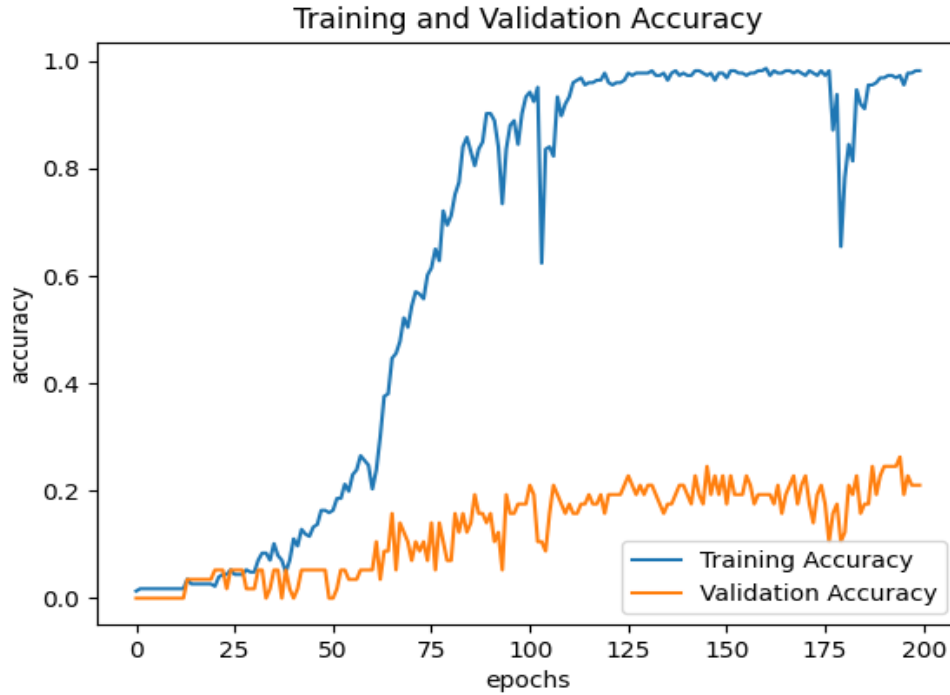


Figure 6.5: Accuracy Curve for Seq2Seq Model

As we can see the Seq2Seq model is heavily overfitting so we have tried another model i.e Transformer Model whose accuracy curve can be shown in figure below:

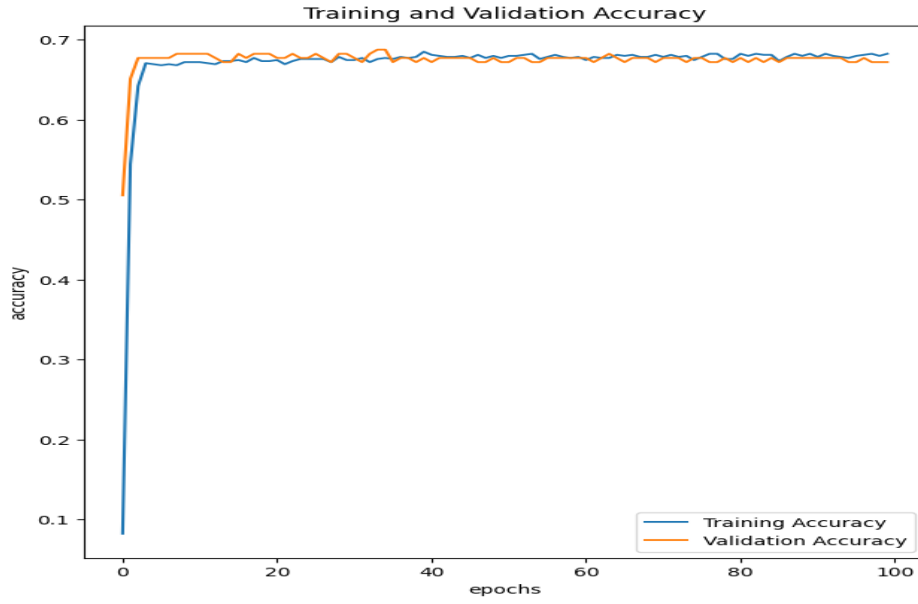


Figure 6.6: Accuracy Curve for Transformer Model

For a language model, accuracy itself is not a reliable metrics to evaluate model's performance. So we used our trained language model to generate predictions for the text samples in our test set. Once we have the probability distribution predicted by our model for each text sample in the test set, we got the perplexity score of 0.271.

Chapter 7

Conclusion

In conclusion, the development and implementation of CompanionCare, our system designed for voice-to-text interaction, has been a comprehensive endeavor integrating diverse technologies and models. Leveraging a CNN1D model, we achieved proficient voice emotion detection, enabling the system to discern users' emotional states in real-time. Employing BERT in conjunction with the Adam optimizer bolstered our text emotion detection capabilities, enabling the system to grasp nuanced sentiments expressed through text inputs. The text for detecting emotion using text was obtained using the voice to text generation module. Using voting mechanism the final emotion was obtained which was then fed to the Generative model.

Intent detection, a critical component of effective communication, was seamlessly integrated using SVM with grid search cross-validation, ensuring accurate identification of user intentions which is then given to the transformer. The utilization of a transformer model for generative responses further elevated the conversational experience, allowing CompanionCare to generate contextually relevant and coherent replies to the user voice input. The text input to transformer was obtained from voice to text conversion module.

The holistic approach of incorporating voice emotion detection, text emotion detection, intent detection, and a generative model has contributed to the creation of a versatile and empathetic conversational agent. CompanionCare stands as a testament to the successful amalgamation of cutting-edge technologies, enabling it to provide a more personalized and engaging user experience. As we look ahead, the robustness of these integrated components positions CompanionCare as an adaptable and scalable solution for diverse applications in the realm of conversational AI.

Chapter 8

Limitations and Future Enhancements

8.1 Limitations

Given the hardware and time constraints and lack of ample dataset there are limitations to our system which are listed below:

- The system currently lacks the capability to generate diverse responses for the same user query. While it excels at providing information and assistance, users may observe a consistent or repetitive pattern in the responses to similar inquiries due to this limitation
- The system's performance is affected by a lack of sufficient training data for the various models employed, leading to a slight reduction in its overall effectiveness.
- Due to the absence of database technology in our system, there is no provision for storing the history of conversations. Consequently, past interactions are not retained or accessible.

8.2 Future Enhancements

The current iteration of our chatbot has demonstrated proficiency in providing response to user queries. However, in the pursuit of continuous improvement, it is essential to identify areas of enhancement. Below ,outlined are the proposed future enhancements thereby advancing the capabilities and user experience.

- We aim to work towards the implementation of advanced algorithms capable of understanding and maintaining context throughout extended dialogues. This enhancement aims to foster coherent and relevant interactions over extended conversations.
- We aim to work the implementation of mechanisms for continuous learning and adaptation. This enhancement enables the system's to refine responses over time, incorporating user feedback to improve overall performance..

- We aim to work towards the integration of support for input in the form of text information and the output in audio form . This enhancement expands the system's versatility, accommodating a broader range of user inputs.

Bibliography

- [1] B. Ford. (2022) quotes. [Online]. Available: <https://medium.com/@brian.ford/our-attitude-towards-life-determines-lifes-attitude-toward-us-9b64cee21f88>
- [2] WHO. (2023) Suicide. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/suicide>
- [3] ——. (2022) Nepal’s national suicide prevention helpline service helping save lives. [Online]. Available: <https://www.who.int/nepal/news/detail/09-12-2022-Nepal-National-Suicide-Prevention-Helpline-Service-helping-save-lives#:~:text=In%20Nepal%2C%20according%20to%20a,increased%20to%2014%25%20in%202021.>
- [4] K. Denecke, S. Vaaheesan, and A. Arulnathan, “A mental health chatbot for regulating emotions (sermo) - concept and usability test,” *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 3, pp. 1170–1182, 2021.
- [5] L. Brocki, G. C. Dyer, A. Gładka, and N. C. Chung, “Deep learning mental health dialogue system,” in *2023 IEEE International Conference on Big Data and Smart Computing (BigComp)*, 2023, pp. 395–398.
- [6] S. Moulya and T. R. Pragathi, “Mental health assist and diagnosis conversational interface using logistic regression model for emotion and sentiment analysis,” *Journal of Physics: Conference Series*, vol. 2161, no. 1, p. 012039, jan 2022. [Online]. Available: <https://dx.doi.org/10.1088/1742-6596/2161/1/012039>
- [7] K.-J. Oh, D. Lee, B. Ko, and H.-J. Choi, “A chatbot for psychiatric counseling in mental healthcare service based on emotional dialogue analysis and sentence generation,” in *2017 18th IEEE International Conference on Mobile Data Management (MDM)*, 2017, pp. 371–375.
- [8] G. Dosovitsky, B. S. Pineda, N. C. Jacobson, C. Chang, and E. L. Bunge, “Artificial intelligence chatbot for depression: Descriptive study of usage,” *JMIR Formative Research*, vol. 4, no. 11, p. e17065, 2020.
- [9] M. R. Ahmed, S. Islam, A. M. Islam, and S. Shatabda, “An ensemble 1d-cnn-lstm-gru model with data augmentation for speech emotion recognition,” *Expert Systems with Applications*, vol. 218, p. 119633, 2023.

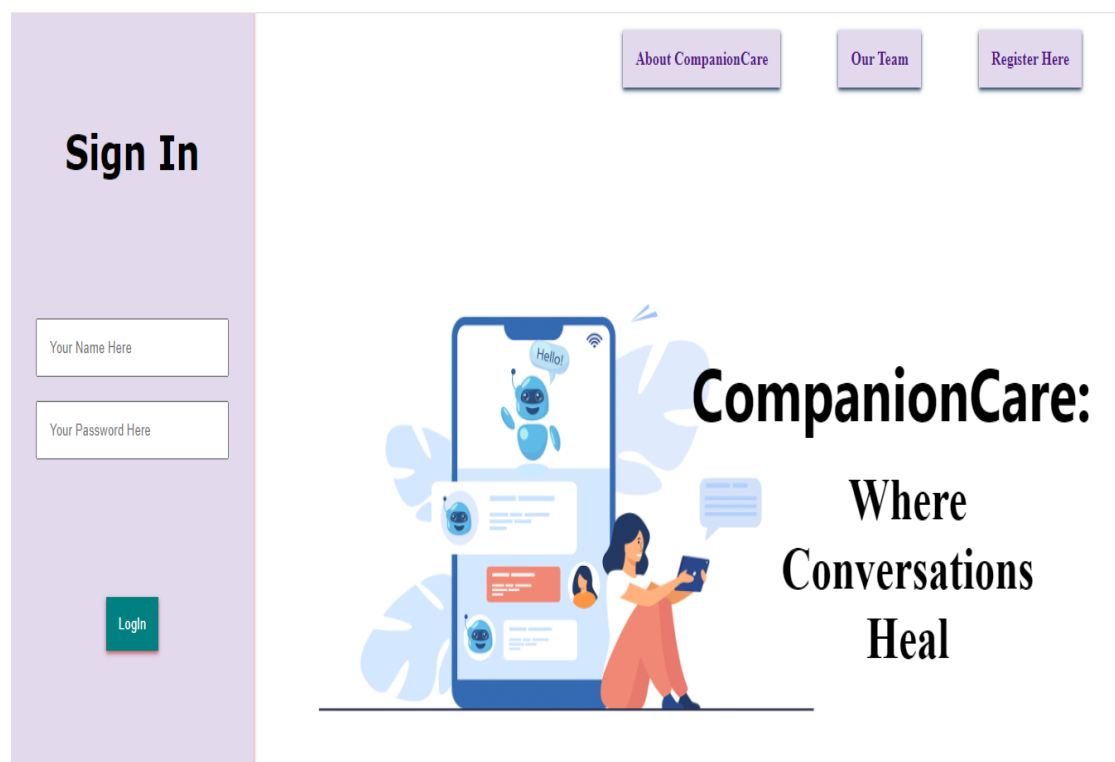
- [10] F. Makhmudov, A. Kutlimuratov, F. Akhmedov, M. S. Abdallah, and Y.-I. Cho, “Modeling speech emotion recognition via attention-oriented parallel cnn encoders,” *Electronics*, vol. 11, no. 23, p. 4047, 2022.
- [11] A. S. Alluhaidan, O. Saidani, R. Jahangir, M. A. Nauman, and O. S. Neffati, “Speech emotion recognition through hybrid features and convolutional neural network,” *Applied Sciences*, vol. 13, no. 8, p. 4750, 2023.
- [12] J. Guo, “Deep learning approach to text analysis for human emotion detection from big data,” *Journal of Intelligent Systems*, vol. 31, no. 1, pp. 113–126, 2022.
- [13] A. Shenfield and M. Howarth, “A novel deep learning model for the detection and identification of rolling element-bearing faults,” *Sensors*, vol. 20, no. 18, p. 5112, Sep. 2020. [Online]. Available: <http://dx.doi.org/10.3390/s20185112>
- [14] “Bert model.” [Online]. Available: <https://towardsdatascience.com/bert-explained-state-of-the-art-language-model-for-nlp-f8b21a9b6270>
- [15] javatpoint. (2022) Support vector machine algorithm. [Online]. Available: <https://www.javatpoint.com/machine-learning-support-vector-machine-algorithm>
- [16] Z. Wang, J. Zhang, and N. Verma, “Realizing low-energy classification systems by implementing matrix multiplication directly within an adc,” *IEEE Transactions on Biomedical Circuits and Systems*, p. 1–1, 2015. [Online]. Available: <http://dx.doi.org/10.1109/TBCAS.2015.2500101>
- [17] T. Zhang, W. Lin, A. M. Vogelmann, M. Zhang, S. Xie, Y. Qin, and J. Golaz, “Improving convection trigger functions in deep convective parameterization schemes using machine learning,” *Journal of Advances in Modeling Earth Systems*, vol. 13, no. 5, May 2021. [Online]. Available: <http://dx.doi.org/10.1029/2020MS002365>
- [18] “Attention is all you need.” [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [19] “Attention layer.” [Online]. Available: <https://machinelearningmastery.com/the-transformer-attention-mechanism/>
- [20] “Feed-forward network.” [Online]. Available: <https://syncedreview.com/2021/04/27/deepmind-podracers-tpu-based-rl-frameworks-deliver-exceptional-performance-at-low-cost-6/>
- [21] S. R. Livingstone and F. A. Russo, “The ryerson audio-visual database of emotional speech and song (ravdess): A dynamic, multimodal set of facial and vocal expressions in north american english,” *PLOS ONE*, vol. 13, no. 5, p. e0196391, May 2018. [Online]. Available: <http://dx.doi.org/10.1371/journal.pone.0196391>

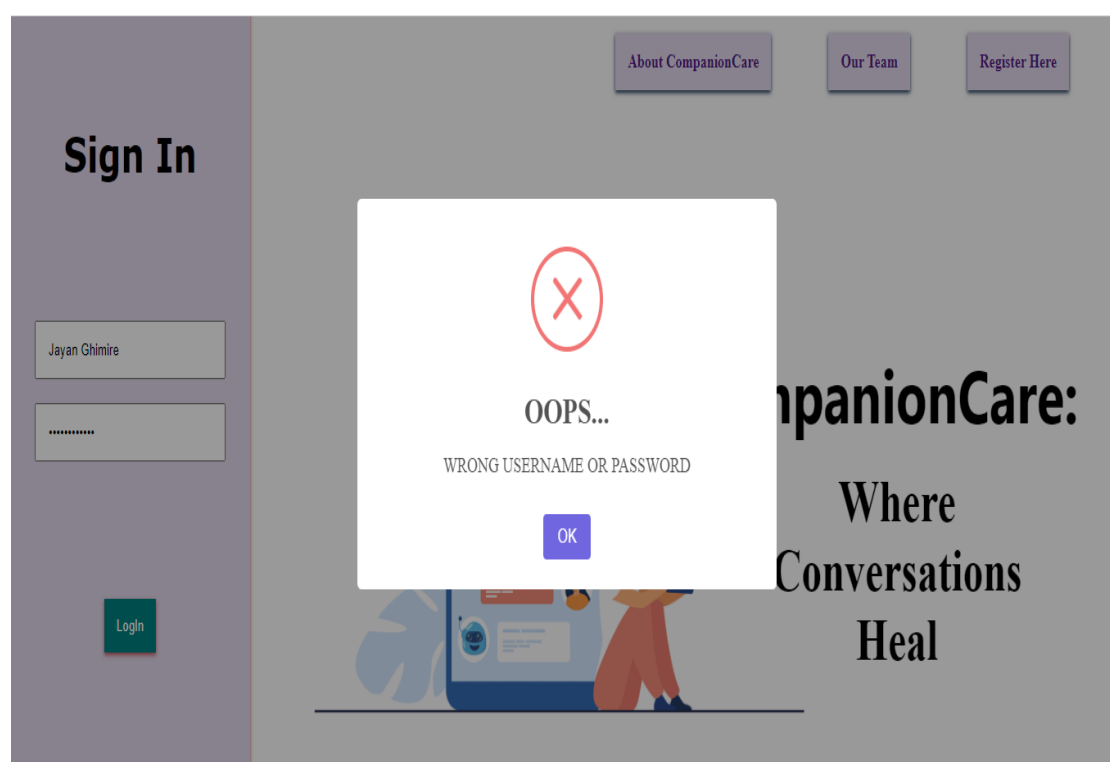
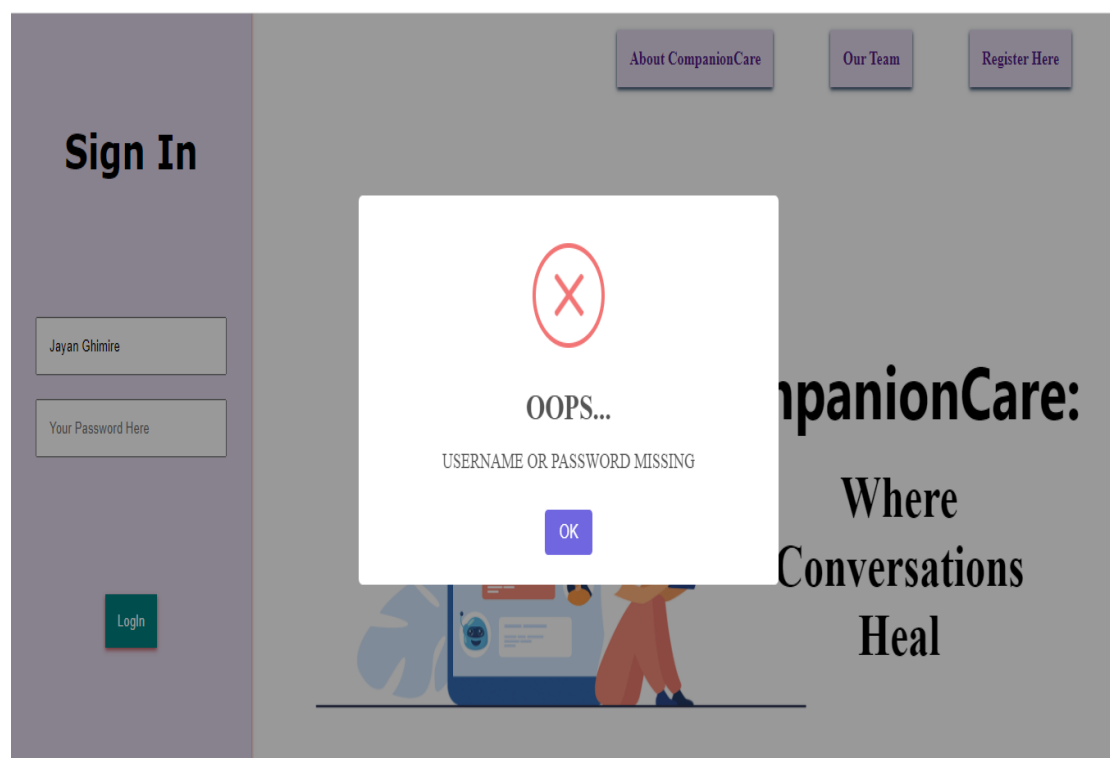
- [22] “Conversational dataset with tag.” [Online]. Available: <https://github.com/pandeyanuradha/Chatbot-for-mental-health/blob/main/Dataset/mentalhealth.json>
- [23] Huggingface. (2022) Empathetic dialogue dataset. [Online]. Available: https://huggingface.co/datasets/empathetic_dialogues
- [24] H. Rashkin, E. M. Smith, M. Li, and Y.-L. Boureau, “Towards empathetic open-domain conversation models: a new benchmark and dataset,” 2018. [Online]. Available: <https://arxiv.org/abs/1811.00207>

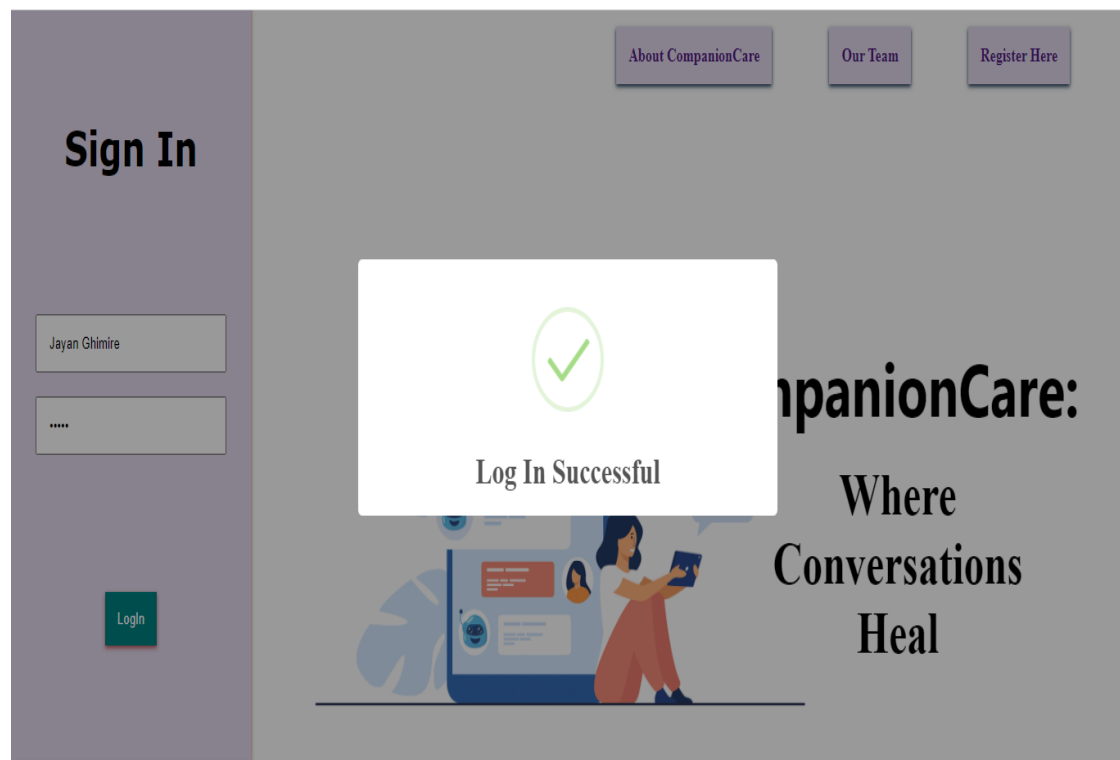
Appendix

C User Interface

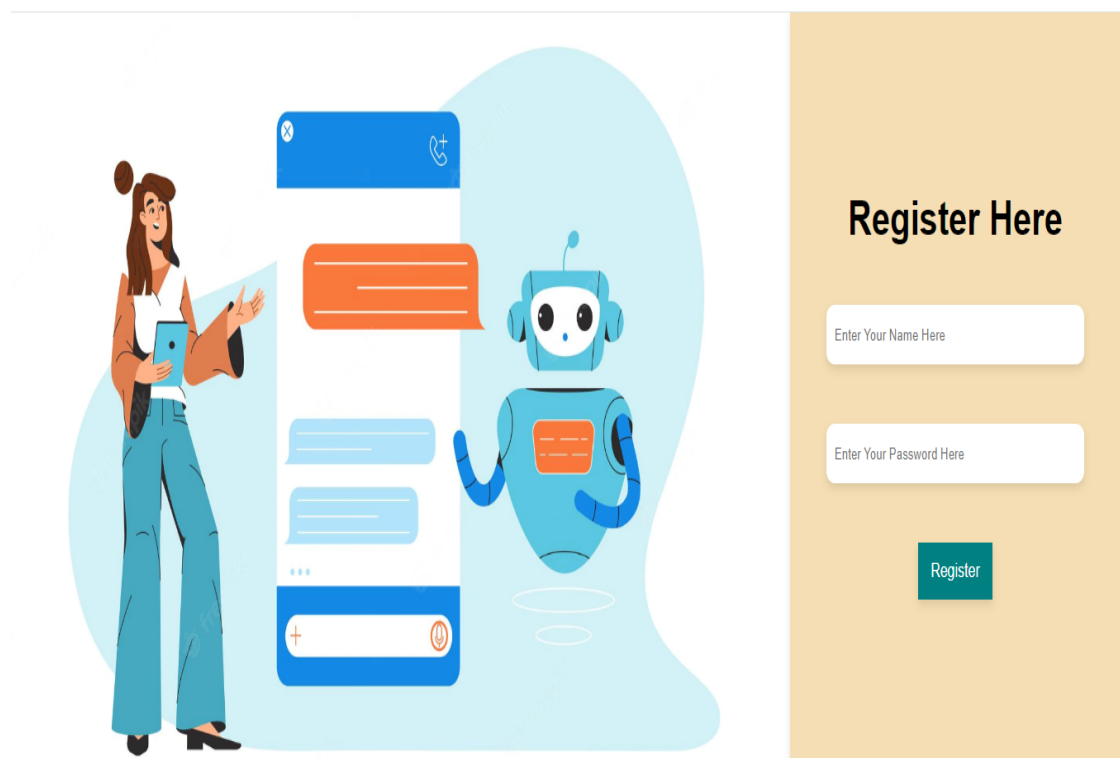
C.1 Home Page

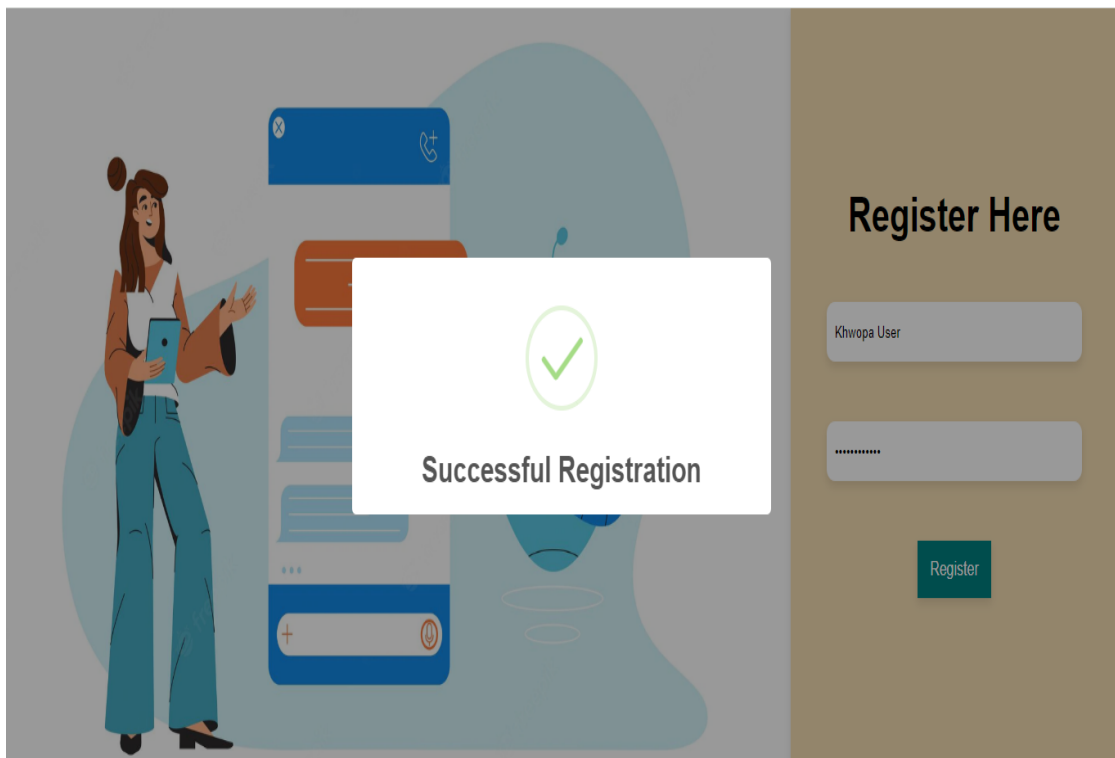
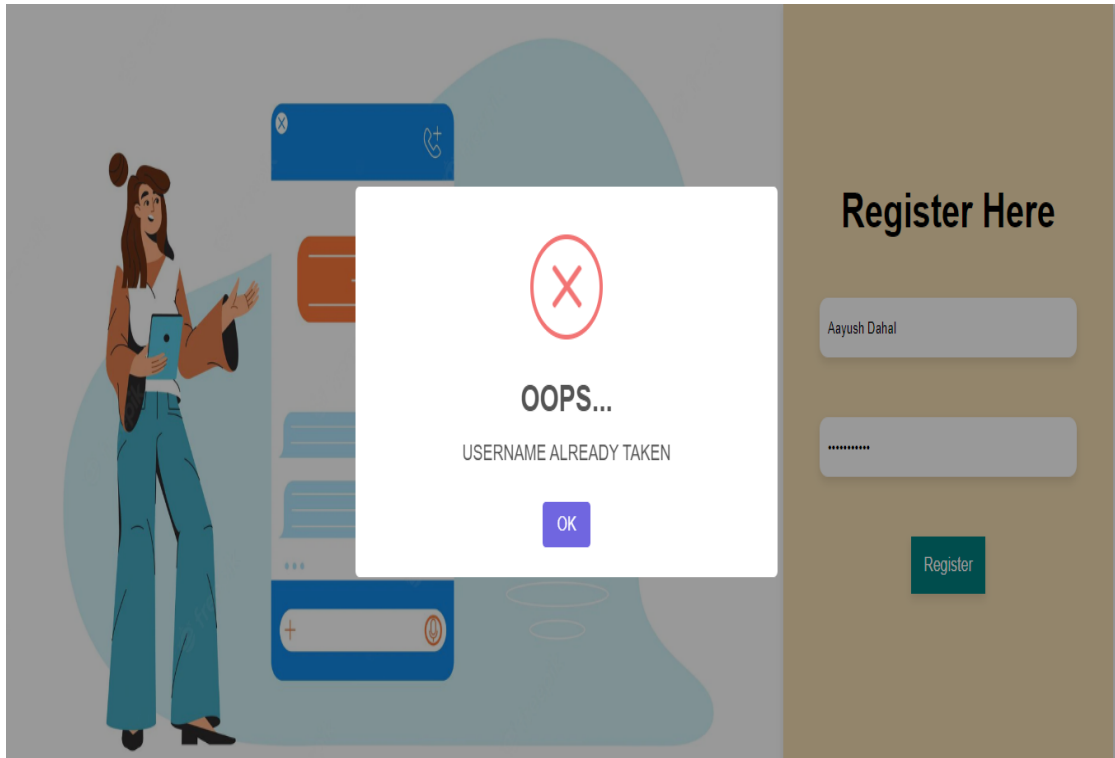




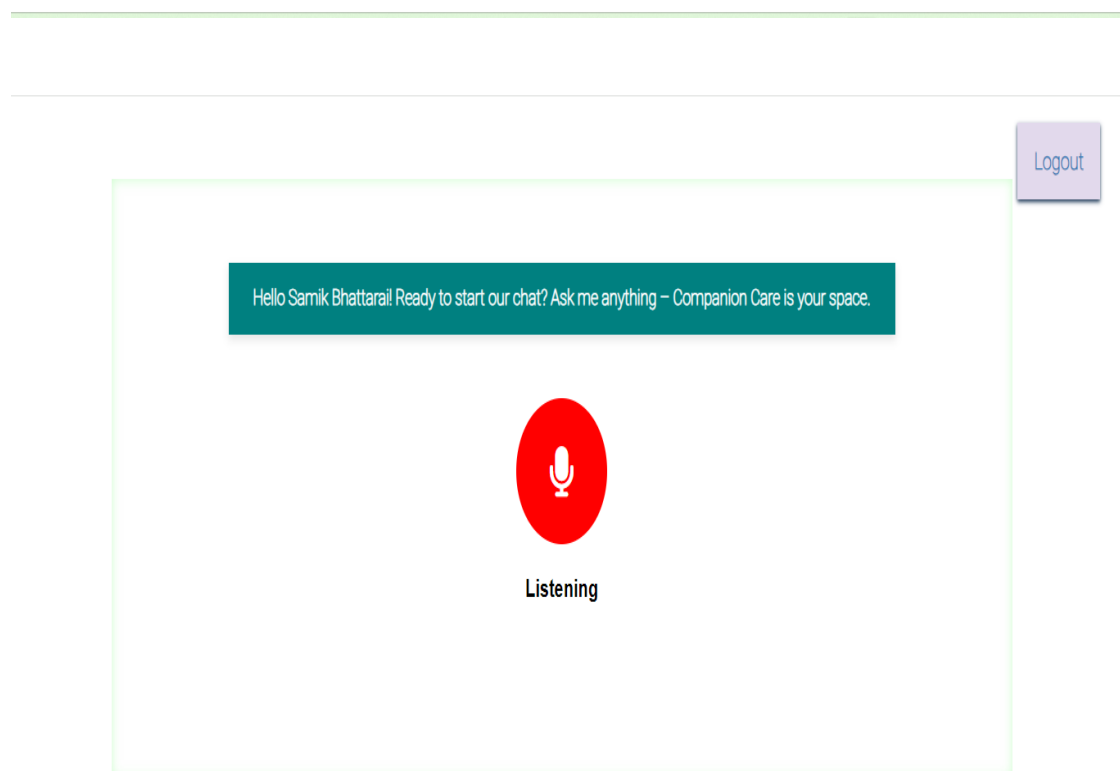
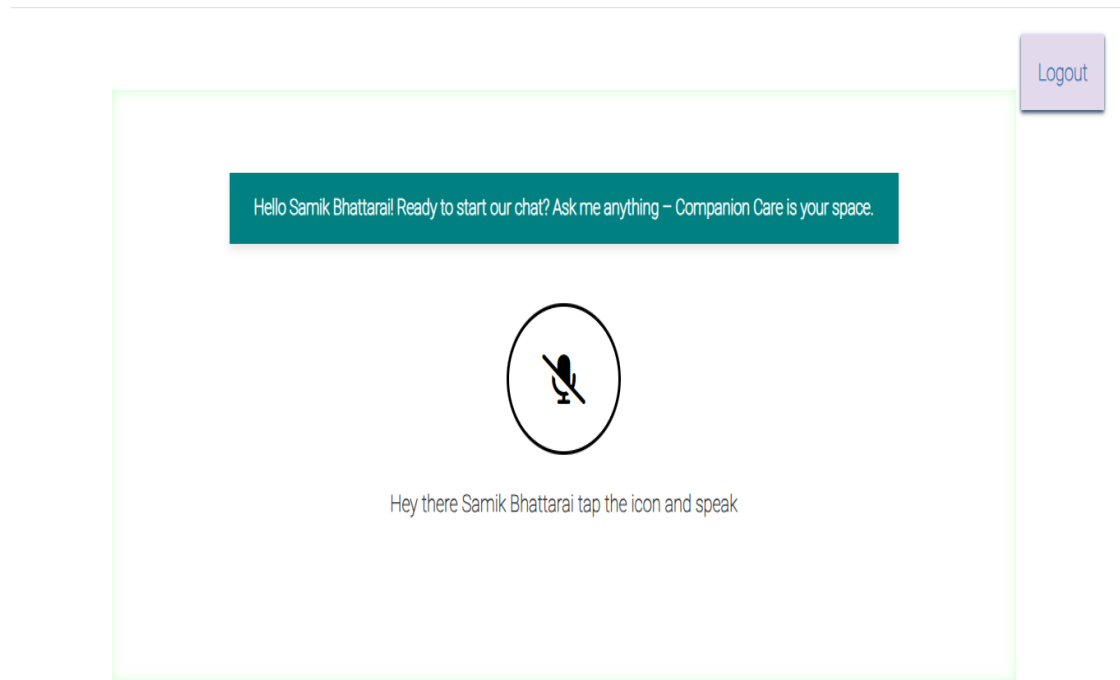


C.2 Register Page





C.3 Chat Demo



CC Hi , how are you ?

I feel like no one understands me JG

CC i am so scared of feeling that , but i hope it will be going to be fine for you

I have no one to talk to JG

CC I think you should be ok right now because you know what s causing the strange feeling

I am starting my school from tomorrow JG

CC That is good news in a long time . Are you going from home ?

yes I am going to go school from my home JG

CC Good I am sure you will be fine . Are you going to go with her ?

yes I think maybe I will go with her JG